STARK-Core-v1

STARK Core Language Specification Overview

Introduction

This document provides an overview of the complete STARK core language specification. The numbered source documents 00–07 plus `CORE-V1-ABSTRACT-MACHINE.md` and `CORE-V1-FUTURE-BOUNDARIES.md` in `docs/spec/` are normative for Core v1. The concise summary and generated combined artifacts are non-normative views, and compiler-governance ledgers under `docs/compiler/semantic-freeze/` are non-normative. The core language defines the general-purpose language surface (lexing, syntax, types, semantics, memory, modules, and standard library). Non-core extensions are defined separately.

Design Philosophy

Core Principles

1. **Memory Safety**: Prevent common memory errors through ownership and borrowing
2. **Performance**: Zero-cost abstractions and predictable execution
3. **Clarity**: Simple, explicit syntax and semantics
4. **Pragmatism**: A minimal, implementable Core v1
5. **Interoperability**: Clear boundaries for future extensions

Language Goals

- A safe, compiled, general-purpose language core
- Compile-time guarantees for memory and type safety
- Simple, predictable semantics suitable for tooling
- Clear extension points for domain-specific features

Specification Structure

1. Lexical Grammar (01-Lexical-Grammar.md)

Defines how source code is tokenized: - **Keywords**: Control flow, declarations, types, operators - **Identifiers**: Variables, functions, types (snake_case/PascalCase) - **Literals**: Integers, floats, strings, characters, booleans - **Operators**: Arithmetic, comparison, logical, bitwise, assignment - **Comments**: Single-line (`//`) and multi-line (`/* */`) - **Whitespace**: Space, tab, newline handling

2. Syntax Grammar (02-Syntax-Grammar.md)

Defines the concrete syntax using EBNF: - **Program Structure**: Items (functions, structs, enums, traits) - **Expressions**: Precedence, associativity - **Statements**: Variable declarations, control flow, returns - **Type Syntax**: Primitives, composites, references, functions - **Pattern Matching**: Destructuring and exhaustiveness

3. Type System (03-Type-System.md)

Comprehensive type system with safety guarantees: - **Primitive Types**: Integers, floats, booleans, characters, strings - **Composite Types**: Arrays, tuples, structs, enums - **Reference Types**: Immutable and mutable references - **Ownership Model**: Move semantics, borrowing rules - **Type Inference**: Local type inference (function signatures are fully explicit) - **Trait System**: Interfaces and generic constraints

4. Semantic Analysis (04-Semantic-Analysis.md)

Rules for meaningful program validation: - **Symbol Resolution**: Scoping, shadowing, name lookup - **Type Checking**: Assignment compatibility, function calls - **Ownership Analysis**: Move tracking, borrow checking - **Control Flow**: Reachability, return path analysis - **Pattern Exhaustiveness**: Match completeness checking - **Error Reporting**: Comprehensive diagnostics with suggestions

5. Memory Model (05-Memory-Model.md)

Memory-safety guarantees and their authority boundaries: - **Ownership Rules**: Single ownership, automatic cleanup - **Move Semantics**: Explicit ownership transfer - **Borrowing System**: Immutable and mutable references - **Lifetime Tracking**: Reference validity guarantees - **Allocation Independence**: Safety rules do not promise a physical allocation strategy - **Drop System**: Deterministic resource cleanup

Abstract Machine (CORE-V1-ABSTRACT-MACHINE.md)

Defines backend-independent execution: - **Values and Places**: Abstract objects, storage, owners, and projections - **Evaluation**: Exact order, exactly-once evaluation, and normal control transfer - **Moves and Destruction**: Replacement, partial moves, temporaries, loops, and collections - **References**: Identity-preserving projection, returned references, and slice views - **Traps and Observations**: Abort boundary and the differential execution comparator

6. Standard Library (06-Standard-Library.md)

Essential types and functions for practical programming: - **Core Types**: Option, Result, Box, Vec, HashMap - **String Handling**: Unicode-aware string operations - **Collections**: Dynamic arrays, hash tables, sets - **IO Operations**: File handling, console output - **Math Functions**: Arithmetic, trigonometric, random numbers - **Error Handling**: Structured error types and propagation

7. Modules and Packages (07-Modules-and-Packages.md)

Defines module structure, visibility, and import resolution: - **Modules**: File and directory layout - **Visibility**: `pub`/`priv` rules - **Imports**: `use` paths, trees, and aliasing - **Packages**: Manifest and dependency resolution

8. Non-Core Extensions (../extensions/)

Non-core language extensions live outside Core v1 and are optional. The tensor & model type system is specified in `docs/extensions/Tensor-Model-Types.md`; the remaining AI/ML surface is sketched in `docs/extensions/AI-Extensions.md`.

Future Boundaries (CORE-V1-FUTURE-BOUNDARIES.md)

Defines the exclusions and compatibility space for capturing closures and explicit lifetimes, concurrency, native providers/FFI, metaprogramming, dynamic dispatch, reserved syntax, and extension isolation. These are boundaries, not implemented Core features.

Core Language Features

Variables and Mutability

```
let x = 42;              // Immutable by default
let mut y = 10;          // Explicitly mutable
const MAX_SIZE: Int32 = 1000;  // Compile-time constant
```

Functions

```
fn add(a: Int32, b: Int32) -> Int32 {
    a + b
}

fn greet(name: &str) {
```

```
    println(name);
}
```

Ownership and Borrowing

```
fn consume(s: String) {
    // s is owned here
}

fn borrow(s: &String) -> UInt64 {
    s.len()
}

fn mutate(s: &mut String) {
    s.push('!');
}
```

This specification provides a focused foundation for implementing a safe, performant, general-purpose language core.

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Lexical Grammar Specification

Overview

This document defines the lexical structure of STARK - how source code is broken down into tokens.

Token Categories

1. Keywords

```
// Control Flow
if, else, match
for, while, loop, break, continue, return

// Declarations
fn, struct, enum, trait, impl, let, mut, const, type, use, mod
```

```
// Types
Int8, Int16, Int32, Int64, UInt8, UInt16, UInt32, UInt64
Float32, Float64, Bool, String, Char, Unit, str

// Visibility
pub, priv

// Module Paths and Self Type
self, Self, super, crate

// Operators
in, as

// Literals
true, false
```

2. Identifiers

```
IDENTIFIER := [a-zA-Z_][a-zA-Z0-9_]*
```

Rules: - Must start with letter or underscore - Can contain letters, digits, underscores - Case sensitive - Cannot be keywords - Maximum length: 255 characters

LEX-IDENT-002. Identifier length is measured in ASCII bytes. Core identifiers are ASCII-only and may contain at most 255 bytes. The lexer must reject an overlength identifier as one token and continue at its end; it may not truncate, split, or intern a colliding prefix.

3. Literals

Integer Literals

```
DECIMAL_INT := [0-9] ('_'? [0-9])*
HEX_INT     := 0[xX] [0-9a-fA-F] ('_'? [0-9a-fA-F])*
BINARY_INT  := 0[bB] [01] ('_'? [01])*
OCTAL_INT   := 0[oO] [0-7] ('_'? [0-7])*

INT_SUFFIX := (i8|i16|i32|i64|u8|u16|u32|u64)

INTEGER := (DECIMAL_INT | HEX_INT | BINARY_INT | OCTAL_INT)
INT_SUFFIX?
```

Rules: - Underscores are digit separators and may appear only *between* two digits — never leading, trailing, or consecutive (`1__2` and `12_` are invalid). - A suffix fixes the literal's type: `i8`→`Int8`, `i16`→`Int16`, `i32`→`Int32`, `i64`→`Int64`, `u8`→`UInt8`, `u16`→`UInt16`, `u32`→`UInt32`, `u64`→`UInt64`. A suffixed literal whose value does not fit the named type is a compile-time error.

Examples:

```
42
1_000_000
```

```
0xFF_FF
0b1010_1010
0o755
42i32
255u8
```

Floating Point Literals

```
FLOAT_BODY := DECIMAL_INT '.' [0-9] ('_'? [0-9])* EXPONENT?
            | DECIMAL_INT EXPONENT
EXPONENT := [eE] [+-]? [0-9] ('_'? [0-9])*

FLOAT_SUFFIX := (f32|f64)

FLOAT := FLOAT_BODY FLOAT_SUFFIX?
```

Rules: - The underscore rules for integer literals apply (separators between digits only). - A suffix fixes the literal's type: `f32`→`Float32`, `f64`→`Float64`.

Examples:

```
3.14
1.0e10
2.5e-3
42.0f32
```

String Literals

```
STRING := '"' (CHAR | ESCAPE_SEQUENCE)* '"'
RAW_STRING := 'r"' .*? '"'

ESCAPE_SEQUENCE := '\' (n|t|r|0|\\|'|"|x[0-9a-fA-F]{2}|u{[0-9a-fA-
F]{1,6}})
```

LEX-ESCAPE-001. `\xNN` contributes exactly one byte and is legal in a string only when the complete decoded string is valid UTF-8; in a character literal it must denote one ASCII scalar. `\u{H...}` must contain one through six hexadecimal digits and denote a Unicode scalar value, excluding surrogate code points and values above U+10FFFF. An invalid escape rejects the literal; no replacement character is inserted.

Examples:

```
"Hello, World!"
"Line 1\nLine 2"
"\x41\x42\x43"
"\u{1F600}"
r"Raw string with \n literal backslashes"
```

Character Literals

```
CHAR := '\'' (CHAR_CONTENT | ESCAPE_SEQUENCE) '\''
CHAR_CONTENT := [^'\\]
```

Examples:

```
'a'
'\n'
'\x41'
'\u{1F600}'
```

Boolean Literals

```
BOOL := true | false
```

4. Operators

Arithmetic

```
+ - * / % **
```

Comparison

```
== != < <= > >=
```

Logical

```
&& || !
```

Bitwise

```
& | ^ ~ << >>
```

Assignment

```
= += -= *= /= %= **= &= |= ^= <<= >>=
```

Range

```
.. ..=
```

Other

```
? :: . -> =>
```

Notes: - `?` is the try operator (postfix). There is no ternary conditional operator; `if` is an expression. - `:` appears only as a delimiter (type annotations, struct fields), not as an operator.

5. Delimiters

```
( ) [ ] { } , ; :
```

6. Comments

```
// Single line comment
/* Multi-line comment */
/** Documentation comment */
```

Rules: - Single line comments start with `//` and continue to end of line - Multi-line comments start with `/*` and end with `*/` - Multi-line comments can be nested - Documentation comments start with `/**` and are associated with following declaration

7. Whitespace

- Space (U+0020)
- Tab (U+0009)
- Newline (U+000A)
- Carriage Return (U+000D)

Whitespace separates tokens and is otherwise ignored. Statement termination is defined by semicolons in the syntax grammar.

8. Token Precedence

When multiple token patterns could match: 1. Keywords take precedence over identifiers 2. Longer operators take precedence over shorter ones 3. Comments are ignored in token stream

9. Reserved Tokens

Reserved for future use (recognized as keywords but not used by any Core v1 grammar production):

```
async, await, yield, where, macro, unsafe, extern, import, export, null,
and, or, not, is, dyn
```

LEX-RESERVED-001. Every spelling in this list tokenizes as a reserved keyword and is rejected wherever Core requires an identifier. An implementation may not treat an unused reserved word as an identifier or silently enable its future meaning.

Lexical Analysis Rules

1. **Maximal Munch**: Always match the longest possible token
2. **Whitespace**: Ignored except for token separation
3. **Encoding**: Source files must be valid UTF-8

LEX-SOURCE-001. A Core source is a sequence of UTF-8 bytes. A byte-order mark at the start is not source whitespace and is rejected. Any invalid UTF-8 sequence rejects the source before tokenization at the first invalid byte offset. Tools may transcode input before presenting it as Core source, but that transport conversion is outside the language and cannot affect source identity or diagnostics for the resulting bytes.

Error Handling

Invalid tokens should produce specific error messages: - Invalid character: "Unexpected character 'X' at line Y:Z" - Unterminated string: "Unterminated string literal at line Y:Z" - Invalid number: "Invalid number format at line Y:Z" ## Conformance A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Syntax Grammar Specification

Overview

This document defines the concrete syntax grammar for STARK using Extended Backus-Naur Form (EBNF).

Grammar Notation

```
::= means "is defined as"
|   means "or" (alternative)
?   means "optional" (zero or one)
*   means "zero or more"
+   means "one or more"
()  means "grouping"
[]  means "optional"
{}  means "zero or more repetitions"
```

Top-Level Grammar

Program

```
Program ::= Item*

Item ::= Visibility? (Function
       | Struct
       | Enum
       | Trait
```

```
        | Impl
        | Const
        | TypeAlias
        | Use
        | Module)

Visibility ::= 'pub' | 'priv'
```

Generic Parameters and Arguments

```
GenericParams ::= '<' GenericParam (',' GenericParam)* ','? '>'

GenericParam ::= IDENTIFIER (':' TraitBounds)?

TraitBounds ::= TraitBound ('+' TraitBound)*

TraitBound ::= Path GenericArgs?

GenericArgs ::= '<' GenericArg (',' GenericArg)* ','? '>'

GenericArg ::= Type
             | IDENTIFIER '=' Type          // Associated type
binding, e.g. Iterator<Item = T>
```

Notes: - Generic parameters may appear on functions, structs, enums, traits, impl blocks, and type aliases. - Generic arguments in *expressions* are inferred at the use site. The only explicit form is `path::<Args>` on a path expression (see `PathExpression`), for calls where inference is impossible, e.g. `size_of::<Int32>()`.

Function Definition

```
Function ::= FunctionSig Block

FunctionSig ::= 'fn' IDENTIFIER GenericParams? '(' ParameterList?
')' ('->' ReturnType)?

ReturnType ::= Type | '!'                    // '!' is the never
type (diverging function)

ParameterList ::= Receiver (',' Parameter)* ','?
                | Parameter (',' Parameter)* ','?

Receiver ::= 'self'
           | '&' 'self'
           | '&' 'mut' 'self'

Parameter ::= IDENTIFIER ':' Type
            | 'mut' IDENTIFIER ':' Type

Block ::= '{' Statement* Expression? '}'
```

Block semantics: - The optional final `Expression` has no terminating semicolon and is the block's value; a block without one evaluates to `Unit`. - Disambiguation: at each position inside a block, the parser first attempts to parse a `Statement`; a trailing `Expression` is recognized only immediately before `}`. An expression followed by `;` is always a statement.

Notes: - A receiver is only valid on functions declared inside a `trait` or `impl` block. - A function without a `->` annotation returns `Unit`. Return types are never inferred.

Data Type Definitions

```
Struct ::= 'struct' IDENTIFIER GenericParams? '{' FieldList? '}'

FieldList ::= Field (',' Field)* ','?

Field ::= IDENTIFIER ':' Type
        | 'pub' IDENTIFIER ':' Type

Enum ::= 'enum' IDENTIFIER GenericParams? '{' VariantList? '}'

VariantList ::= Variant (',' Variant)* ','?

Variant ::= IDENTIFIER
          | IDENTIFIER '(' TypeList ')'
          | IDENTIFIER '{' FieldList '}'

Trait ::= 'trait' IDENTIFIER GenericParams? '{' TraitItem* '}'

TraitItem ::= FunctionSig ';'               // Required method
(signature only)
            | FunctionSig Block             // Method with default
body
            | AssociatedType

AssociatedType ::= 'type' IDENTIFIER ';'

Impl ::= 'impl' GenericParams? Type '{' ImplItem* '}'
       | 'impl' GenericParams? TraitBound 'for' Type '{' ImplItem*
'}'

ImplItem ::= Visibility? Function
           | 'type' IDENTIFIER '=' Type ';' // Associated type
value
```

Module Definition

```
Module ::= 'mod' IDENTIFIER ';'
        | 'mod' IDENTIFIER ModuleBlock

ModuleBlock ::= '{' Item* '}'
```

Type Alias

```
TypeAlias ::= 'type' IDENTIFIER GenericParams? '=' Type ';'
```

Statements

```
Statement ::= ';'                       // Empty statement
            | Expression ';'            // Expression statement
            | BlockExpression ';'?      // Block-formed expression
statement
            | 'let' LetStatement
            | 'return' Expression? ';'
            | 'break' Expression? ';'
            | 'continue' ';'

BlockExpression ::= Block
                  | IfExpression
                  | MatchExpression
                  | LoopExpression

LetStatement ::= IDENTIFIER ':' Type ';'
               | 'mut' IDENTIFIER ':' Type ';'
               | IDENTIFIER ':' Type '=' Expression ';'
               | IDENTIFIER '=' Expression ';'
               | 'mut' IDENTIFIER ':' Type '=' Expression ';'
               | 'mut' IDENTIFIER '=' Expression ';'
```

Block-formed expression statements: - An expression whose outermost form is a block, `if`, `match`, `loop`, `while`, or `for` may stand as a statement without a terminating semicolon (`if ready { start(); }` mid-block is a statement). - Statement parsing is greedy: when a block-formed expression begins at statement position and is not followed by `;`, it is complete as a statement — a following token does NOT extend it into a larger expression (`{ if c { } - 1; }` is an `if` statement followed by an error, not a subtraction). Parenthesize to use the value: `(if c { 1 } else { 2 }) - 1`. - Exception (trailing expression): a block-formed expression immediately before the closing `}` of its block and not followed by `;` is the block's trailing `Expression` (its value), not a statement — so `fn max(...) -> T { if a > b { a } else { b } }` returns the `if`'s value.

Expressions

```
Expression ::= AssignmentExpression

AssignmentExpression ::= RangeExpression
                       | RangeExpression AssignOp
AssignmentExpression

AssignOp ::= '=' | '+=' | '-=' | '*=' | '/=' | '%=' | '**='
           | '&=' | '|=' | '^=' | '<<=' | '>>='
```

Place Expressions

The left-hand side of an assignment must be a *place expression* — an expression that denotes a memory location. Place expressions are:

SYN-PLACE-001. The forms in `PlaceExpression` are the complete Core v1 assignment-target syntax. Parentheses do not change whether an expression is a place. Field, tuple-field, index, and dereference forms are accepted by the parser and then require the mutable-place typing rules in `03-Type-System.md`; calls, literals, casts, ranges, aggregates, and other computed values are never assignment targets.

```
PlaceExpression ::= Path                              // Variable
                  | PlaceExpression '.' IDENTIFIER    // Field
                  | PlaceExpression '.' INTEGER       // Tuple
field
                  | PlaceExpression '[' Expression ']' // Index
                  | '*' Expression                    //
Dereference
                  | '(' PlaceExpression ')'
```

Assignment to any other expression form is a compile-time error (checked semantically; the grammar accepts any expression on the left).

```
RangeExpression ::= LogicalOrExpression (('..' | '..=')
LogicalOrExpression)?

LogicalOrExpression ::= LogicalAndExpression ('||'
LogicalAndExpression)*

LogicalAndExpression ::= EqualityExpression ('&&'
EqualityExpression)*

EqualityExpression ::= RelationalExpression (('==' | '!=')
RelationalExpression)?

RelationalExpression ::= BitwiseOrExpression (('<' | '<=' | '>' |
'>=') BitwiseOrExpression)?

BitwiseOrExpression ::= BitwiseXorExpression ('|'
BitwiseXorExpression)*

BitwiseXorExpression ::= BitwiseAndExpression ('^'
BitwiseAndExpression)*

BitwiseAndExpression ::= ShiftExpression ('&' ShiftExpression)*

ShiftExpression ::= AdditiveExpression (('<<' | '>>')
AdditiveExpression)*

AdditiveExpression ::= MultiplicativeExpression (('+' | '-')
MultiplicativeExpression)*

MultiplicativeExpression ::= ExponentiationExpression (('*' | '/'
| '%') ExponentiationExpression)*
```

```
ExponentiationExpression ::= CastExpression ('**'
ExponentiationExpression)?

CastExpression ::= UnaryExpression ('as' Type)*

UnaryExpression ::= PostfixExpression
                  | '-' UnaryExpression
                  | '!' UnaryExpression
                  | '~' UnaryExpression
                  | '&' 'mut'? UnaryExpression // Reference
(immutable or mutable borrow)
                  | '*' UnaryExpression        // Dereference

PostfixExpression ::= PrimaryExpression
                    | PostfixExpression '(' ArgumentList?
')'     // Function call
                    | PostfixExpression '[' Expression
']'        // Index (or slice, when the index is a range)
                    | PostfixExpression '.'
IDENTIFIER            // Field access / method reference
                    | PostfixExpression '.'
INTEGER               // Tuple access
                    | PostfixExpression
'?'                       // Try operator

ArgumentList ::= Expression (',' Expression)* ','?

PrimaryExpression ::= PathExpression
                    | Literal
                    | '(' Expression ')'    // Grouping
                    | TupleLiteral
                    | ArrayLiteral
                    | StructLiteral
                    | IfExpression
                    | MatchExpression
                    | LoopExpression
                    | Block

PathExpression ::= Path ('::' GenericArgs)?  // e.g. x,
String::from, Color::Red, size_of::<Int32>
```

Notes: - `path::<Args>` (explicit generic arguments on a path expression) is permitted only when type inference cannot determine the arguments, e.g. `size_of::<Int32>()`. It is the only form of expression-level generic arguments in Core v1. - Chained comparisons (`a < b < c`, `a == b == c`) are syntax errors: the relational and equality productions are non-associative (at most one operator). Parenthesize to compare a Bool result explicitly.

Control Flow Expressions

```
IfExpression ::= 'if' Expression Block ('else' 'if' Expression
Block)* ('else' Block)?
```

```
MatchExpression ::= 'match' Expression '{' MatchArm* '}'

MatchArm ::= Pattern '=>' Expression ','?

Pattern ::= Literal
          | '_'                                    // Wildcard
          | IDENTIFIER                             // Binding (or
unit variant/const, see note)
          | Path                                   // Unit enum
variant or const, e.g. Color::Red
          | Path '(' PatternList? ')'              // Tuple enum
variant, e.g. Option::Some(x)
          | Path '{' FieldPatternList? '}'         // Struct or
struct enum variant
          | '(' PatternList? ')'                   // Tuple
          | '[' PatternList? ']'                   // Array

PatternList ::= Pattern (',' Pattern)* ','?

FieldPatternList ::= FieldPattern (',' FieldPattern)* ','?

FieldPattern ::= IDENTIFIER ':' Pattern
               | IDENTIFIER                        // Shorthand:
binds field to same-named variable

LoopExpression ::= 'loop' Block
                 | 'while' Expression Block
                 | 'for' IDENTIFIER 'in' Expression Block
```

SYN-PATTERN-001. The productions above are the complete Core v1 pattern surface. Core v1 has no pattern guards, alternative (`|`) patterns, rest patterns, binding-mode annotations, or range patterns. A named-field pattern may list fields in any order but may not repeat a field. Static typing, exhaustiveness, usefulness, and ownership are defined by `04-Semantic-Analysis.md` and `CORE-V1-ABSTRACT-MACHINE.md`.

Note on pattern name resolution: a single `IDENTIFIER` pattern that resolves to a unit enum variant or a constant in scope matches by value; otherwise it introduces a new binding. Multi-segment `Path` patterns always match by value.

Literals

```
Literal ::= INTEGER
          | FLOAT
          | STRING
          | CHAR
          | BOOLEAN

TupleLiteral ::= '(' ')'                                   // Unit
value
               | '(' Expression ',' ')'                    //
Single-element tuple
```

```
               | '(' Expression (',' Expression)+ ','? ')' // N-element tuple

ArrayLiteral ::= '[' ExpressionList? ']'
               | '[' Expression ';' Expression ']'        // Repeat: [value; count]

ExpressionList ::= Expression (',' Expression)* ','?

StructLiteral ::= Path '{' FieldInitList? '}'

FieldInitList ::= FieldInit (',' FieldInit)* ','?

FieldInit ::= IDENTIFIER ':' Expression
            | IDENTIFIER                    // Shorthand: field: field
```

Notes: - `(expr)` is grouping; `(expr,)` is a single-element tuple; `()` is the unit value. The trailing comma distinguishes the 1-tuple from grouping. - In `[value; count]`, `count` must be a compile-time constant expression of an unsigned integer type, and `value`'s type must implement `Copy` (the value is copied `count` times).

Struct Literal Restriction

To avoid ambiguity with block-taking constructs, a struct literal may NOT appear as the outermost expression in: - the condition of an `if` or `while`, - the scrutinee of a `match`, - the iterable of a `for`.

Wrap the struct literal in parentheses in these positions:

```
if (Config { verbose: true }).verbose { do_thing(); }
```

Types

```
Type ::= PrimitiveType
       | Path GenericArgs?                   // Named type, possibly generic: Vec<Int32>, Option<T>
       | '[' Type ';' INTEGER ']'            // Array type
       | '[' Type ']'                        // Slice type (unsized; used behind references)
       | TupleType
       | '(' Type ')'                        // Grouping (the type T, not a tuple)
       | '&' Type                            // Reference type
       | '&' 'mut' Type                      // Mutable reference type
       | 'fn' '(' TypeList? ')' ('->' Type)?  // Function type (non-capturing)

TupleType ::= '(' ')'                        // Unit type
            | '(' Type ',' ')'               // Single-element tuple type
            | '(' Type (',' Type)+ ','? ')' // N-element tuple
```

```
type

PrimitiveType ::= 'Int8' | 'Int16' | 'Int32' | 'Int64'
                | 'UInt8' | 'UInt16' | 'UInt32' | 'UInt64'
                | 'Float32' | 'Float64'
                | 'Bool' | 'Char' | 'String' | 'Unit' | 'str'

TypeList ::= Type (',' Type)* ','?
```

Notes: - A function type without `->` returns `Unit`. - The never type `!` may appear only as a function return type (see `ReturnType`). - `(T)` is the type `T` (grouping); the single-element tuple type is `(T,)`, mirroring `TupleLiteral` on the value side.

Other Constructs

```
Const ::= 'const' IDENTIFIER ':' Type '=' Expression ';'

Use ::= 'use' UseTree ';'

UseTree ::= Path ('as' IDENTIFIER)?
          | Path '::' '*'
          | Path '::' 'self'
          | Path '::' '{' UseTreeList? '}'

UseTreeList ::= UseTree (',' UseTree)* ','?

Path ::= PathSegment ('::' PathSegment)*
PathSegment ::= IDENTIFIER | PrimitiveType | 'self' | 'Self' |
'super' | 'crate'
```

Notes: - `Self` is valid only inside a `trait` or `impl` block, and only as the *first* segment of a path. As a complete one-segment path in type position it denotes the implementing type; with further segments it projects an associated item, e.g. `Self::Item`. - `self`, `super`, and `crate` are likewise valid only as leading segments (`self`/`super` may repeat at the front: `super::super::x`). - A `PrimitiveType` keyword is valid only as the *first* segment, where it names the primitive's associated items: `String::from(s)`. (Primitive type names are keywords in the lexical grammar, so without this rule such calls would be unparseable.)

Operator Precedence (Highest to Lowest)

1. Primary expressions, field access, array access, function calls, try operator (`?`)
2. Unary operators (-, !, ~, &, &mut, *)
3. Cast (`as`)
4. Exponentiation (**)
5. Multiplicative (*, /, %)
6. Additive (+, -)
7. Shift (<<, >>)
8. Bitwise AND (&)
9. Bitwise XOR (^)
10. Bitwise OR (|)

11. Relational (<, <=, >, >=)
12. Equality (==, !=)
13. Logical AND (&&)
14. Logical OR (||)
15. Range (.., ..=)
16. Assignment (=, +=, -=, etc.)

Associativity

- Left associative: Most binary operators
- Right associative: Assignment operators, exponentiation
- Non-associative: Comparison operators, range operators

Statement vs Expression

- Statements do not return values and end with semicolons (block-formed expression statements may omit the semicolon — see Statements)
- Expressions return values and can be used as statements with semicolons
- Blocks are expressions (return value of last expression, or Unit if none)
- Control flow constructs are expressions

Whitespace and Semicolons

- Semicolons are required to terminate statements, except after block-formed expression statements (see Statements)
- Semicolons are optional after the last expression in a block
- Newlines are not significant except for line comments
- Trailing commas are allowed in lists

Parsing Notes

SYN-RECOVERY-001. For Core conformance, parsing observes only acceptance or rejection and the required primary diagnostic category/location. Token insertion/deletion, synchronization points, cascaded diagnostics, partial syntax trees, and recovery continuation are implementation-defined tooling behavior. Recovery must not accept a source that the grammar rejects or reject a source that the grammar accepts.

- When a > is expected in generic-argument position and the next token is >>, >>=, or >= (maximal munch), the parser MUST split off a single > and re-tokenize the remainder (>, >=, or = respectively) — so `Vec<Vec<Int32>>`, `Vec<Vec<Int32>>= v`, and `Vec<Int32>= v` all parse.
- Nested tuple-field access `pair.0.1` lexes the `0.1` as a FLOAT token (maximal munch); after `.`, the parser MUST split an unsuffixed digits-`.`-digits FLOAT into two INTEGER tuple indices.

- In expression position, a bare < after an identifier is always the relational operator; explicit generic arguments require the `::<` form (turbofish), so no lookahead disambiguation is required.

Informative Future Grammar Sketches (Non-Normative)

Reserved grammar constructs for later implementation:

```
// Async functions (future)
AsyncFunction ::= 'async' 'fn' IDENTIFIER '(' ParameterList? ')' ('->' Type)? Block

// Lambda expressions / capturing closures (future)
Lambda ::= '|' ParameterList? '|' (Expression | Block)

// Open-ended ranges (future)
OpenRange ::= Expression '..' | '..' Expression | '..'

// Lifetime annotations (future)
LifetimeParam ::= '\'' IDENTIFIER
```

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Type System Specification

Overview

STARK features a static type system with type inference, ownership semantics, and memory safety guarantees.

Core Principles

1. **Static Typing**: All types resolved at compile time
2. **Type Inference**: Types can be inferred when unambiguous
3. **Memory Safety**: No null pointer dereferences, no use-after-free
4. **Ownership**: Clear ownership and borrowing rules
5. **Zero-cost Abstractions**: Type safety without runtime overhead

Primitive Types

Integer Types

```
Int8    // 8-bit signed integer (-128 to 127)
Int16   // 16-bit signed integer (-32,768 to 32,767)
Int32   // 32-bit signed integer (-2^31 to 2^31-1)
Int64   // 64-bit signed integer (-2^63 to 2^63-1)

UInt8   // 8-bit unsigned integer (0 to 255)
UInt16  // 16-bit unsigned integer (0 to 65,535)
UInt32  // 32-bit unsigned integer (0 to 2^32-1)
UInt64  // 64-bit unsigned integer (0 to 2^64-1)
```

Default integer type is `Int32` for literals that fit, `Int64` otherwise.

Floating Point Types

```
Float32  // 32-bit IEEE 754 floating point
Float64  // 64-bit IEEE 754 floating point
```

Default floating point type is `Float64`.

Other Primitive Types

```
Bool    // Boolean: true or false
Char    // Unicode scalar value (32-bit)
Unit    // Unit type: () - represents no meaningful value
str     // String slice (unsized), typically used behind
references: &str
```

String Type

```
String  // UTF-8 encoded string, heap-allocated, growable
```

String Slice Type

```
// str is an unsized string slice type. It is used via references.
let s: &str = "hello";
let owned: String = String::from(s);
```

Never Type

`!` (the never type) is the type of expressions that never produce a value, such as calls to `panic`. It may appear only as a function return type.

```
fn panic(message: &str) -> !
```

Rules: - An expression of type `!` coerces to any other type. This allows, for example, `panic(...)` to appear in one arm of an `if` or `match` whose other arms produce a value. - A function returning `!` MUST NOT return normally.

Core Type Identity

TYPE-PRIM-001. Each named primitive type is distinct. `Unit` and `()` are two spellings of the same single-inhabitant type. Array identity is the ordered pair of element type and length; tuple identity is its ordered type sequence; reference identity includes pointee type and mutability; function identity includes the ordered parameter types and return type. The never type `!` is uninhabited and distinct from every other type.

Composite Types

Array Types

```
[T; N]   // Fixed-size array of N elements of type T (sized)
[T]      // Slice of elements of type T (unsized)
```

A slice `[T]` is an *unsized* view into a contiguous sequence (an array, or the elements of a `Vec<T>`). Like `str`, it is used behind references:

```
let fixed: [Int32; 5] = [1, 2, 3, 4, 5];
let view: &[Int32] = &fixed;          // Array reference coerces
to slice reference
let part: &[Int32] = &fixed[1..4];    // Slicing with a range
yields &[T]
```

A local variable cannot have the bare unsized type `[T]`; use `[T; N]` for a fixed-size array or `Vec<T>` (standard library) for a growable one.

Indexing rules: - `expr[i]` where `i` is an integer denotes a *place* of type `T` (bounds-checked; traps on out-of-bounds). - `expr[r]` where `r` is a `Range` denotes a *place* of the unsized slice type `[T]`; traps if the range is out of bounds or inverted. Because `[T]` is unsized, a slice place must be borrowed immediately: `&expr[r]` has type `&[T]` and `&mut expr[r]` has type `&mut [T]`.

Range Type

The range operators produce values of the standard library `Range<T>` type:

```
let r = 0..10;      // Range<Int32>: 0,1,...,9 (half-open)
let ri = 0..=9;     // RangeInclusive<Int32>: 0,1,...,9
```

Ranges over integer types implement `Iterator` and may be used with `for` loops and slicing.

Tuple Types

```
()             // Empty tuple (Unit type)
(T,)           // Single-element tuple
(T1, T2)       // Two-element tuple
(T1, T2, T3) // Three-element tuple
// ... up to 16 elements
```

Examples:

```
let empty: () = ();
let single: (Int32,) = (42,);
let pair: (Int32, String) = (42, "hello");
```

Struct Types

```
struct Point {
    x: Float64,
    y: Float64
}

struct Person {
    name: String,
    age: Int32,
    pub email: String  // Public field
}
```

Restriction (Core v1): struct and enum declarations MUST NOT declare fields whose *written* type is a reference type (`&T`, `&mut T`). Instantiating a generic type with a reference type argument (e.g. `Option<&T>`) is permitted; the resulting value is *borrow-carrying* — see “References and Lifetimes” below.

TYPE-NOMINAL-001. A struct or enum type has nominal identity:

```
canonical package instance + module path + item name + normalized
generic arguments
```

The canonical package-instance token is supplied by the package identity rules completed in C2.9. Relocation, import aliases, re-exports, and local type aliases do not change identity. Two separately declared types remain distinct even when their fields and variants are structurally identical.

Enum Types

```
enum Color {
    Red,
    Green,
    Blue
}

enum Option<T> {
    Some(T),
    None
```

```
}

enum Result<T, E> {
    Ok(T),
    Err(E)
}
```

Reference Types

```
&T       // Immutable reference to T
&mut T   // Mutable reference to T
```

Function Types

Function types are written with the `fn` keyword and denote *non-capturing* functions (named functions or function references). Capturing closures are a future extension.

```
fn(T1, T2) -> R    // Function taking T1, T2 and returning R
fn() -> R          // Function taking no parameters, returning R
fn(T)              // Function taking T, returning Unit
```

TYPE-FN-001. Function values are `Copy` and `Clone` (see Copy and Drop) and never implement `Drop`. In Core v1, function types do **not** implement `Eq`, `Ord`, or `Hash`: comparing or hashing function values is a compile-time error, exactly as for the floating-point primitives. Because no Core v1 program can compare or hash a function value, function-value *identity* is not observable; implementations must guarantee only that calling a function value invokes the function it was created from. A future extension may add comparison by function identity; it must then also define the observable identity of monomorphised generic instances.

TYPE-FN-002. A generic function may be used where a concrete function type is expected only when the expected type fully determines every generic argument. The conversion instantiates the generic function at those arguments; the resulting value is the monomorphised instance, and calling it behaves exactly like calling the generic function with those type arguments. If the expected function type does not fully determine the generic arguments (or the bounds are not satisfied by them), the program is rejected at compile time.

Type Aliases

```
type Age = Int32;
type Point2D = (Float64, Float64);
type ErrorCode = Int32;
```

TYPE-ALIAS-001. A type alias is transparent. After capture-avoiding substitution of its generic arguments, it denotes exactly its target type and introduces no nominal identity, constructor, implementation-coherence boundary, or distinct method set. Alias expansion is recursive; an alias cycle is a compile-time error. A nominal wrapper requires a distinct struct or enum declaration.

Type Well-Formedness and Sizedness

TYPE-WF-001. Every value type and every generic parameter in Core v1 is implicitly `Sized`; Core has no syntax to relax this bound. The only unsized types are implementation-provided `str` and slice `[T]`, and they may occur only immediately behind `&` or `&mut`. Bare unsized locals, parameters, returns, fields, tuple elements, array elements, generic arguments, and `Box<str>`/`Box<[T]>` are prohibited in Core v1.

A declaration is infinitely sized when its field-type dependency graph has a cycle containing only direct value edges. References and the implementation-provided owning indirections `Box<T>` and `Vec<T>` break that graph. Direct and indirect value recursion is rejected; recursion through one of those indirections is well formed when all other rules hold.

The following edge types are explicit:

- `[T; 0]` is a sized, inhabited empty array and still requires sized `T`;
- empty structs are sized, inhabited nominal types;
- zero-variant enums are sized, uninhabited nominal types;
- tuples support arities zero through sixteen; `()` is `Unit`;
- array length is a constant `UInt64`; Core imposes no smaller semantic maximum, while finite compiler/target resource limits are classified by C2.9;
- `!` is uninhabited, may be written only as a function return type, and coerces to any expected type.

```
struct InvalidNode { next: InvalidNode }
struct ValidNode { next: Option<Box<ValidNode>> }
type Meter = Int32;
```

The Self Type

Within a `trait` or `impl` block, `Self` refers to the implementing type:

```
trait Eq {
    fn eq(&self, other: &Self) -> Bool;
}
```

Ownership and Borrowing

Ownership Rules

1. Each value has exactly one owner
2. When the owner goes out of scope, the value is dropped
3. Values can be moved (ownership transfer) or borrowed (temporary access)

Move Semantics

```
let a = String::from("hello");
let b = a;  // a is moved to b, a is no longer valid
// print(a);  // Error: use of moved value
```

Borrowing Rules

OWN-BORROW-001. For any overlapping place, a live exclusive borrow prohibits every other read, write, borrow, move, or destruction through a different access path; one or more live shared borrows permit other shared reads/borrows but prohibit writes, exclusive borrows, moves, and destruction. Disjoint field projections do not overlap. Index projections are treated as overlapping unless the checker proves distinct constant indices. Reborrowing cannot grant stronger mutability or outlive the source reference.

1. You can have either one mutable reference or any number of immutable references
2. References must always be valid (no dangling pointers)
3. References cannot outlive the data they refer to

```
fn borrow_immutable(s: &String) {
    // Can read but not modify
}

fn borrow_mutable(s: &mut String) {
    // Can read and modify
}

let mut text = String::from("hello");
borrow_immutable(&text);       // OK
borrow_mutable(&mut text);     // OK
```

References and Lifetimes (Core v1)

Core v1 has no lifetime annotations. Instead, it enforces the following conservative rules, which are normative:

OWN-REGION-001. A reference or borrow-carrying value bound to a local remains live from creation through the end of that local's lexical block, unless the complete carrier is consumed earlier by `drop`. Moving a carrier transfers, rather than ends, its live borrow: the borrow then remains live through the destination carrier's lexical region or through completion of the consuming call. A reference parameter is live for the call. An unbound borrow used as a subexpression remains live through its enclosing full expression; borrowed rvalue arguments and borrowed temporary method receivers therefore remain live through call completion. Core v1 performs no last-use shortening and no temporary-lifetime extension for `let r = &make_value()`; such an escaping temporary reference is rejected.

OWN-RETURN-001. A returned reference or borrow-carrying value must derive, on every return path, only from reference parameters. Its caller-visible region is the intersection (the shortest region) of every reference parameter from which that path's result may derive. Projections into the referent of a reference parameter are permitted. A local, temporary, by-value parameter, constant temporary, or a field whose root is one of those non-reference sources cannot be returned by reference.

1. **No declared reference fields.** Struct, enum variant, and tuple struct declarations MUST NOT write a reference type (`&T`, `&mut T`) as a field type. (Generic *instantiation* with a reference argument is allowed; see Borrow-Carrying Types below.)
2. **References derive from parameters.** A function may return a reference (or borrow-carrying value) only if every control-flow path derives it from one of its reference parameters (directly, by field/index projection, or by calling a function that itself obeys this rule). Returning a reference derived from a local variable or a by-value parameter is a compile-time error.
3. **Shortest-input-lifetime rule.** The lifetime of a returned reference (or borrow-carrying value) is the *shortest* of the lifetimes of all reference parameters from which it could have been derived. Callers MUST treat it as invalidated as soon as the shortest-lived of those arguments is invalidated.
4. **Local borrows are lexically scoped.** A borrow bound to a variable (`let r = &x;`) lasts from its creation to the end of its enclosing block (or until the borrower is explicitly dropped with `drop(...)`). The borrow checker enforces the exclusive/shared rules over that entire region — even if the reference is never used again. To end such a borrow early, introduce an inner block or call `drop`. A *temporary* borrow that is not bound to a variable (e.g. `f(&x);`) ends at the end of its enclosing statement, so `f(&x); g(&mut x);` is legal.

```
let mut s = String::from("hello");
{
    let r = &s;           // Borrow begins
    println(r.as_str());
}                         // Borrow ends with the block
s.push('!');              // OK: no live borrow
```

Example of rule 3:

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
// The returned reference is valid only while BOTH x's and y's
referents
// are valid, regardless of which branch was taken.
```

These rules reject some safe programs (that is intentional). Lifetime annotations that relax rule 3, user structs with declared reference fields, and non-lexical (last-use) borrow regions are future extensions; because the lexical rule is strictly more conservative, adopting last-use regions later cannot break conforming programs.

Borrow-Carrying Types

OWN-CARRY-001. Borrow provenance is structural and transitive through tuples, generic arguments, enum payloads, function arguments/results, pattern bindings, and implementation-provided borrowed views. A merge of control-flow paths carries the union of possible source referents and the most restrictive capability; its region cannot exceed the intersection of their valid regions.

A type is **borrow-carrying** if it is: - a reference type `&T` or `&mut T`; or - a generic type with at least one borrow-carrying type argument (e.g. `Option<&T>`, `(Int32, &str)`); or - an implementation-provided borrowed view type (the standard library's borrowed iterators such as `VecIter<T>`, `CharsIter`, `KeysIter<K>`, and the slice types behind references).

A borrow-carrying value is treated *as if it were a reference* for all rules in this section: - It counts as a live borrow of the value(s) it was derived from — shared for `&`-derived values, exclusive for `&mut`-derived values — for the borrow region defined by rule 4. - It may instantiate a generic user-declared field (for example `Holder<&Int32>` where the declaration writes `value: T`). The complete instantiated aggregate becomes borrow-carrying with the same sources and capability. This does not permit a declaration to write `&T` directly. - It may be nested in other generic aggregates without a depth restriction, but every enclosing value remains borrow-carrying and may be returned only under rules 2–3 or otherwise used only within the lifetime of every source. - Binding it with `let` is permitted; the binding is subject to rule 4.

This is a taint rule: borrow-carrying-ness propagates through generic instantiation and function returns, and the checker tracks the source variables each borrow-carrying value may borrow from. It is what makes APIs like `Vec::get(&self, i) -> Option<&T>` sound without lifetime annotations: the returned `Option<&T>` is an active shared borrow of the `Vec` until it goes out of scope.

Type Inference

Deterministic Local Inference

TYPE-INFER-001. Inference is local to one function or constant body and never infers item signatures. For each unannotated local, the checker collects equality and trait constraints from its initializer and every use of that binding until its scope ends or it is shadowed. Consequently, later uses within that region may constrain the local, but uses outside it and bodies of called functions may not.

Expected types flow inward from explicit annotations, function parameters, return types, assignment destinations, aggregate fields, branch/arm unification, and an enclosing call's expected result. Constraint collection is independent of traversal or declaration-map iteration order. Solving proceeds as follows:

1. expand transparent aliases and create fresh variables for every generic instantiation;
2. collect exact-type equations, expected-type constraints, associated-type obligations, and required trait bounds;
3. repeatedly apply equality unification and uniquely selected associated-type normalization to a fixed point;
4. apply permitted coercions only at their explicit coercion sites;
5. default an unconstrained integer literal to `Int32` when representable, otherwise `Int64`, and an unconstrained float literal to `Float64`;
6. reject any remaining unconstrained variable, conflicting equation, unsatisfied obligation, or non-unique solution as an inference error.

Inference never chooses an implementation or overload merely because it was declared first. Error-recovery variables may suppress cascaded diagnostics but must not turn a rejected program into an accepted one.

```
let x = 42;             // Inferred as Int32
let y = 3.14;           // Inferred as Float64
let z = [1, 2, 3];      // Inferred as [Int32; 3]
```

Function Return Types

Function return types are NOT inferred. A function without a -> annotation returns `Unit`.

```
fn add(a: Int32, b: Int32) -> Int32 {
    a + b
}

fn greet(name: &str) {      // Returns Unit
    println(name);
}
```

Generic Type Inference

TYPE-GENERIC-001. Each generic use receives fresh parameters. Explicit turbofish arguments bind the corresponding parameters first; omitted arguments are inferred from value arguments and then from the expected result type. All occurrences of one parameter must normalize to one type. Associated type bindings are obligations, not new inference variables, and normalize only after unique trait selection. If any parameter remains unconstrained, the call requires explicit arguments.

Generic declarations are compared after alpha-renaming parameters and capture-avoiding substitution. Bounds are conjunctions independent of source order. Monomorphization and dictionary passing are implementation strategies, not type-system differences.

```
fn identity<T>(x: T) -> T {
    x
}

let result = identity(42);  // T inferred as Int32
```

Subtyping and Coercion

Core v1 has no general subtyping. At an expected-type boundary, coercions are attempted in this deterministic order: identity/alias normalization, never coercion, mutable-to-shared reference weakening, then array-reference to slice-reference coercion. A coercion may combine mutable-to-shared weakening with array-to-slice exactly once (`&mut [T; N]` to `&[T]`); no other coercion chains, implicit numeric conversions, user conversions, or trait-based coercions exist.

Numeric Coercions

No implicit numeric conversions. Explicit casting required:

```
let x: Int32 = 42;
let y: Int64 = x as Int64;  // Explicit cast required
```

Cast semantics (`as`): - Between integer types: value-preserving if in range; a cast whose value does not fit the target type is a runtime error and MUST trap. - Between floating point types: rounds to nearest representable value. - Integer to float and float to integer: float-to-int truncates toward zero and MUST trap if the result does not fit the target type (including NaN/Inf).

TYPE-CAST-001. `as` is legal only between numeric primitive types. Its exact checked conversion result is `NUM-CAST-001`. A cast never invokes a trait, allocates, changes ownership, or silently wraps. A statically known failing cast is a compile-time error; otherwise the required failure is a language trap at the cast expression.

Reference Coercions

```
&mut T -> &T        // Mutable reference to immutable reference
&T -> &T            // Same type (identity)
```

Array to Slice Coercion

TYPE-COERCE-003. Array-to-slice coercion is available only for references. It preserves the original referent, bounds, and borrow region; it never copies elements or constructs an owned slice. Shared input produces shared output. Mutable input may produce mutable output or be weakened to shared output.

```
&[T; N] -> &[T]     // Array reference to slice reference
&mut [T; N] -> &mut [T]  // Mutable array reference to mutable slice reference
```

Never Coercion

```
! -> T              // The never type coerces to any type
```

Trait System (Basic)

Trait Definition

TRAIT-DEF-001. A trait declares required methods, optional default method bodies, and associated types. An implementation must define every associated type and required method not supplied by a default, with an alpha-equivalent generic signature, receiver form, parameter types, return type, and bounds. An implementation may override a default but may not add trait-associated items. Trait and implementation bodies are type-checked under their declared bounds.

```
trait Display {
    fn fmt(&self) -> String;
}

trait Eq {
    fn eq(&self, other: &Self) -> Bool;
}
```

Trait Implementation

```
impl Display for Int32 {
    fn fmt(&self) -> String {
        // Convert integer to string
    }
}

impl Eq for Point {
    fn eq(&self, other: &Point) -> Bool {
        self.x == other.x && self.y == other.y
    }
}
```

Associated Types

TRAIT-ASSOC-001. An associated item is identified by its declaring trait and item name. `T::Item` in a trait-bound context is a projection, not a free name; it normalizes only after a unique applicable implementation is selected. Fully qualified syntax selects the named trait. A missing associated value, conflicting binding, unresolved projection, or normalization cycle is a compile-time error.

Traits may declare associated types; implementations assign them:

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}

struct Counter { count: Int32 }

impl Iterator for Counter {
    type Item = Int32;
    fn next(&mut self) -> Option<Int32> {
        self.count += 1;
        Some(self.count)
    }
}
```

Bounds may constrain associated types with bindings: `I: Iterator<Item = T>`.

Type Checking Rules

Assignment Compatibility

```
let x: T = expr;  // expr must have type T or be coercible to T
```

Function Call Compatibility

```
fn f(param: T) { ... }
f(arg);  // arg must have type T or be coercible to T
```

Arithmetic Operations

```
// Binary arithmetic requires same types
let result = x + y;  // x and y must have the same numeric type

// Comparison operators
let cmp = x < y;     // x and y must have the same comparable type
```

Logical Operations

```
let both = a && b;    // a and b must be Bool
let negated = !x;     // x must be Bool
```

For Loops

`for x in expr` requires `expr` to have a type that implements `Iterator`; the loop variable binds successive `Item` values. Integer ranges implement `Iterator`; slices and collections provide `.iter()` methods (see the standard library specification).

```
for i in 0..10 {
    println(i.fmt());
}
```

Control Flow Typing

- **`if`/`else`**: an `if` with an `else` is an expression whose branches must unify to a single type (identical types, or unifiable via the `!` coercion — e.g. one branch may `panic`). An `if` *without* `else` has type `Unit`, and its branch must also have type `Unit`.
- **`match`**: all arm expressions must unify to a single type (with `!` coercion permitted per arm).
- **`loop`**: a `loop` expression's type is the unified type of the operands of its `break value;` statements; if every `break` is bare (no value) or the loop has no `break`, the type is `Unit` (a `loop` with no `break` has type `!`).
- **`while` / `for`**: always have type `Unit`. `break` inside `while`/`for` MUST NOT carry a value.
- **Block**: the type of the trailing expression, or `Unit` if there is none.

TYPE-LOOP-001. Reachable `break` operands in one `loop` are unified using the ordinary expected-type and never-coercion rules. A reachable bare `break` contributes `Unit`. A `loop` with no reachable `break` has type `!`; otherwise it has the unique unified break type. `while` and `for` have type `Unit` and reject value-carrying `break`.

Method Calls and Auto-Borrowing

TYPE-METHOD-001. Method selection is independent of declaration and hash-map iteration order. For each receiver type considered by auto-dereference, the checker collects every visible, type-applicable inherent candidate. If that set is nonempty it must contain exactly one candidate and trait candidates are ignored. Otherwise it collects applicable methods from traits in lexical scope or the prelude; exactly one must remain after generic substitution and obligations. Zero candidates is unresolved and multiple candidates is ambiguous. Fully qualified `Trait::method(receiver, ...)` bypasses trait-name lookup but still requires a unique coherent impl.

A method call `recv.m(args)` is resolved as follows:

1. **Candidate collection.** Candidates are, in priority order:
   1. inherent methods — `fn m` in `impl RecvType { ... }` blocks;
   2. trait methods — `fn m` from any trait that RecvType implements, where the trait is *in scope* (defined in, or imported via `use` into, the current module, or in the prelude). Inherent methods shadow trait methods of the same name.
2. **Ambiguity.** If two or more traits in scope supply an applicable `m` and no inherent method exists, the call is a compile-time error. Disambiguate with a fully-qualified call, passing the receiver explicitly: `TraitName::m(&recv, args)`.
3. **Receiver coercion (auto-borrowing).** Let `S` be the receiver expression's type. Candidates are matched trying these receiver forms in order: `self: S` (by value), `self: &S` (auto-borrow: the compiler inserts `&`), `self: &mut S` (auto-mutable-borrow: the compiler inserts `&mut`; requires the receiver to be a mutable place). Auto-borrows follow the normal borrow rules.
4. **Auto-dereference.** If `S` is `&T` or `&mut T` and no candidate matches on `S`, resolution retries with `T` (one level of dereference, applied repeatedly for nested references). Combined with step 3 this allows `(&v).len()` and `v.len()` to resolve identically.
5. **Visibility.** Private methods are callable only per the module rules (07-Modules-and-Packages.md).

Trait methods can always be called in fully-qualified function form — `Display::fmt(&x)` — since a method is an ordinary function whose first parameter is the receiver.

TYPE-METHOD-002. Auto-dereference examines `S`, then repeatedly removes one leading `&`/`&mut`; at each level receiver matching tries by-value, shared-borrow, then exclusive-borrow form. Selection stops at the first dereference level with an applicable candidate. Auto-borrow never evaluates the receiver twice, never moves through a shared reference, and may create `&mut` only from a mutable place not conflicting with a live borrow. No argument-position auto-borrow or auto-dereference exists. Argument expressions may still undergo the closed set of built-in expected-type coercions defined in "Subtyping and Coercion" (the reference coercions and TYPE-COERCE-003); a function parameter is an expected-type boundary like any other. No user-defined coercion exists.

Operators and Traits

Operator expressions on **primitive types** have built-in meaning (Numeric Semantics below). Equality and ordering on every non-primitive concrete type, and on generic parameters, require one uniquely selected coherent `Eq` or `Ord` implementation and dispatch to its method. Arithmetic on a generic parameter requires `Num` and becomes the corresponding primitive operation after monomorphization:

Operator	Non-primitive/generic requirement	Meaning
`==`, `!=`	`T: Eq`	`Eq::eq(&a, &b)` (negated for `!=`)
`<`, `<=`, `>`, `>=`	`T: Ord`	`Ord::cmp(&a, &b)` compared to `Ordering`
`+ - * / %`, bitwise, shifts	generic `T: Num`	primitive operation after monomorphization
`**`	integer primitive type only	checked integer exponentiation

Which **primitive** types admit which of these operators, and which implement `Eq`/`Ord`/`Hash` for the purpose of generic bounds, is specified normatively by `PRIM-TRAIT-001` in 06-Standard-Library ("Primitive Trait and Operator Matrix"). The two questions are distinct for primitives: operators on primitives have built-in meaning and do not dispatch through the traits, so `Float64` admits < and == as IEEE operations while satisfying neither `T: Ord` nor `T: Eq`. `Bool` admits ==/`!=` but no ordered operator; `Char` is ordered by Unicode scalar value.

`Num` is a compiler-known marker trait implemented by exactly the built-in numeric types (`Int8`–`Int64`, `UInt8`–`UInt64`, `Float32`, `Float64`); user types cannot implement it in Core v1. `Num` does not by itself authorize `**`: floating exponentiation uses `std::math::pow`, while the operator is integer-only. Operator overloading for user-defined types (Add/Sub/… traits) is a future extension. &&, `||`, and `!` (on `Bool`) are built-in, short-circuiting, and not overloadable.

```
fn max<T: Ord>(a: T, b: T) -> T {
    if a > b { a } else { b }   // a > b desugars to Ord::cmp(&a,
&b)
}
```

Evaluation Order (Core v1)

Runtime evaluation order is defined solely by `CORE-V1-ABSTRACT-MACHINE.md` (`EXEC-EVAL-001`, `EXEC-ONCE-001`, and `EXEC-ASSIGN-001`). Type checking must preserve those rules and must not introduce an evaluation or ownership transfer merely to select a type, method, or trait implementation.

Copy and Drop (Soundness Rules)

OWN-COPY-001. — Copy eligibility.

A type is `Copy` if it is one of the language-defined `Copy` primitives (the built-in scalar primitives except `String`, `Unit`, `!`, and function values), a shared reference, or a compound/nominal type whose stored fields are recursively `Copy` and whose type has no `Drop` behavior.

Copy eligibility may be derived structurally for nominal user types (struct and enum) only when: - **the nominal has at least one field-bearing form**: a struct always qualifies (a fieldless struct has exactly one value), but an **enum with zero variants is never structurally `Copy`**; - every stored field/payload is recursively `Copy`; - no field is an owned non-`Copy` resource (`Box`, `Vec`, `String`, maps, sets); - no field requires `Drop` glue; - the nominal itself has no `Drop` implementation or destructor behavior; - no field is a mutable reference (exclusive references are never `Copy`; they would participate only if the core type system explicitly admitted them, which it does not).

On the zero-variant enum clause (amended, prompted by CD-234). Without it the rule reaches the wrong answer by vacuous truth: "every payload of every variant is `Copy`" is trivially satisfied when there are no variants. That reasoning silently assumes every value of the type arose from one of those variants.

A zero-variant enum is used precisely as an **opaque nominal whose values may enter from outside the source language** — no expression and no pattern can manufacture one, which is what makes it opaque. Once such a value exists, duplicating it is observable and unsound: for a host resource it means two owners of one handle and a close that runs twice.

The clause is a general language rule, not a resource-specific exception. It costs nothing elsewhere: no program can rely on copying a value of an uninhabited type, because no program can obtain one by ordinary construction.

Generic nominals are `Copy` per-instance under the corresponding `T: Copy` obligations: `struct H<T>` is `Copy` when instantiated at a `Copy` `T` and not otherwise. Arrays and tuples are `Copy` exactly when every component is.

`Copy` is inferred structurally for eligible non-dropping user nominals; no explicit implementation is required. An explicit `Copy` implementation, where supported, MUST NOT admit any type that fails this same recursive eligibility predicate. A type that IS `Copy` remains usable after assignment, argument passing, return, or pattern binding; a type that is NOT `Copy` is moved by those operations unless borrowed. Pattern binding is the one case where "unless borrowed" is decided by the *scrutinee* rather than by the pattern: a component matched through a reference is borrowed, not moved — see `PAT-BIND-001` in `04-Semantic-Analysis.md`.

- `Copy` may be implemented for a type only if **all** of its fields are `Copy`. Violations are compile-time errors.
- A type MUST NOT implement both `Copy` and `Drop` (a copyable type would run its destructor once per copy).
- `Drop::drop` MUST NOT be called explicitly; use the free function `drop(value)`. After `drop(v)`, `v` is moved-from.

- **Partial moves**: moving a field out of a struct is permitted only if the struct's type does not implement `Drop`. After a partial move, the whole value may no longer be used or moved, but its remaining fields may be.
- **Reinitialization**: assigning a new value to a moved-from `let mut` variable makes it valid again (definite-assignment tracking).
- Moving an element out of an indexed place (`v[i]`) is a compile-time error; use APIs that transfer ownership explicitly (e.g. `Vec::remove`, `Vec::pop`).

The execution and ordering of moves, copies, replacement, `drop(value)`, partial-value destruction, and trap termination are defined solely by `CORE-V1-ABSTRACT-MACHINE.md`.

Error Types and Handling

Option Type

```
enum Option<T> {
    Some(T),
    None
}
```

Result Type

```
enum Result<T, E> {
    Ok(T),
    Err(E)
}
```

Error Propagation

```
fn might_fail() -> Result<Int32, String> {
    // ...
}

fn caller() -> Result<Int32, String> {
    let value = might_fail()?;  // Early return on error
    Ok(value * 2)
}
```

Try Operator (?) Typing

The try operator is defined for Result<T, E> and Option<T>: - If expr has type Result<T, E>, then expr? has type T and propagates Err(E) to the nearest enclosing function returning Result<_, E>. - If expr has type Option<T>, then expr? has type T and propagates None to the nearest enclosing function returning Option<_>.

The enclosing function's return type must be compatible with the propagated type.

Generics (Core v1)

Core v1 supports parametric polymorphism for functions, structs, enums, and traits.

Rules: - Generic parameters are introduced with `<T, U, ...>` after the item name. - Generic parameters are in scope within the item body and signatures. - All generic parameters used in an item MUST be declared by that item. - Instantiation occurs at use sites; Core v1 permits monomorphization or dictionary-passing, but the observable behavior MUST be equivalent. - Generic arguments in expressions are inferred. When inference is impossible (no parameter or usable result type mentions the type parameter), explicit arguments are supplied with the turbofish form on a path expression: `size_of::<Int32>()`. This is the only expression-level generic-argument syntax in Core v1.

```
struct Pair<T> {
    first: T,
    second: T
}

fn max<T: Ord>(a: T, b: T) -> T {
    if a > b { a } else { b }
}
```

Trait Bounds

```
fn max<T: Ord>(a: T, b: T) -> T { if a > b { a } else { b } }
```

Rules: - A bound `T: Trait` requires exactly one applicable coherent `impl Trait for T` in the resolved package graph; trait visibility controls whether its name may be written, not whether an existing obligation can be proven. - Multiple bounds are allowed: `T: TraitA + TraitB`. - Bounds may bind associated types: `I: Iterator<Item = T>`.

Trait Coherence (Core v1)

TRAIT-COHERENCE-001. A trait implementation is permitted only when the current canonical package instance defines either the trait or the head nominal type after transparent-alias expansion. Type parameters, primitives, tuples, arrays, references, and function types are not local head types. Different selected package versions have distinct nominal identities; import aliases and re-exports do not make a trait or type local. Inherent implementations are permitted only for a nominal type defined by the current package.

TRAIT-COHERENCE-002. Coherence is checked over the complete resolved package graph, independent of source order and whether an implementation is used. Two implementations overlap whenever a substitution unifies their trait and self-type heads. Positive trait bounds never make unifying heads disjoint: without negative bounds or sealed-world traits, they may simultaneously hold. Disjointness is proved only by incompatible nominal type constructors, unequal concrete types/

constant generic arguments, different trait identities, or recursively disjoint type arguments. Duplicate and overlapping implementations are rejected before obligation selection.

Selection for a concrete obligation:

1. expand aliases and normalize already-resolved projections;
2. collect all coherent impl heads that unify with the obligation;
3. instantiate each with fresh variables and recursively prove its positive bounds;
4. require exactly one successful candidate;
5. substitute its associated types and continue normalization.

Recursive obligation cycles that do not establish a strictly smaller, previously proven obligation are rejected. Core v1 has no specialization, negative implementations, trait objects, or declaration-order priority. Blanket implementations are permitted only when the overlap test proves them disjoint from every other implementation.

Standard Trait Laws

TRAIT-LAW-001. Implementations used as Core `Eq`, `Ord`, and `Hash` must obey these semantic laws:

- `Eq::eq` is reflexive, symmetric, and transitive;
- `Ord::cmp` defines a total order, returns `Equal` exactly when `Eq::eq` is true, and is antisymmetric and transitive;
- values equal under `Eq` produce equal `Hash::hash` results;
- hashing an unchanged value is stable for the complete execution of one program under one fixed target/environment.

The compiler is required to reject only violations it can prove; it is not required to prove arbitrary method bodies lawful. Direct calls still execute the user bodies normally. If an implementation violates a law, operations whose contract relies on that law—including ordered comparison and hash collections—have unspecified logical results, but memory safety, ownership, trap behavior, and exactly-once destruction remain guaranteed. A law violation never grants undefined behavior. C2.9 separately decides which floating types, if any, implement these traits.

Constant Evaluation (Core v1)

CONST-DECL-001. A `const` declaration is evaluated at compile time and must produce a value exactly matching or coercible to its declared type. Array-repeat counts and constant patterns use the same evaluator; array-type lengths remain the integer literals required by the Core grammar and are validated at compile time. Constants are immutable values, not addressable storage locations: each value-context use produces the constant value, and borrowing a constant materializes an ordinary temporary governed by the C2.7 temporary rules. Declaration order does not affect visibility or evaluation.

CONST-SUBSET-001. Constant expressions are the following closed, side-effect-free subset:

- literals, unit, and references to other constants;

- grouping; tuples; arrays; array repeats; and struct or enum construction;
- primitive unary, arithmetic, bitwise, shift, logical, comparison, and range operations;
- `if` expressions whose condition and selected branch are constant;
- explicit primitive casts;
- blocks containing only constant expressions and a final constant value.

Every operand follows the abstract-machine evaluation order. Transparent aliases, generic substitutions, and expected types are resolved before evaluation. Function or method calls, trait dispatch, local bindings, assignment, borrowing/dereference, indexing requiring runtime storage, `match`, loops, `return`/`break`/`continue`, `?`, `panic`, heap-backed collections, I/O, artifact/native-provider access, and runtime state are prohibited. No user function is implicitly treated as a constant function.

Floating literals are allowed. Other floating constant operations are allowed only where C2.9 supplies a deterministic operation/cast result; the constant evaluator must then produce exactly that runtime result and may not use host extended precision or host-language defaults.

CONST-FAIL-001. Constant dependencies form a graph across the resolved package graph. Any dependency cycle is a compile-time error with the cycle reported deterministically. A prohibited form, type mismatch, ambiguous inference, overflow, division/remainder by zero, invalid shift, invalid cast, bounds failure, or any other operation that would trap at runtime is instead a compile-time constant-evaluation error at the failing operation. Both branches are type-checked, but only the selected branch of a constant `if` is evaluated.

Evaluation is deterministic and memoization may not change diagnostics or results. Finite implementation resource limits are governed by C2.9; reaching one must produce its classified diagnostic and must never be cached as a semantic constant value.

Numeric Semantics (Core v1)

NUM-FLOAT-FORMAT-001. `Float32` is IEEE 754 binary32 and `Float64` is IEEE 754 binary64. All value observations use those formats; an implementation may use wider intermediates only when it rounds to the declared format at every operation boundary and produces the same result.

NUM-FLOAT-TRAIT-001. `Float32` and `Float64` implement `Copy`, `Clone`, `Default`, and `Display`, but not `Eq`, `Ord`, or `Hash`. Primitive == and ordering comparisons remain IEEE comparisons: NaN compares unequal to every value including itself, every ordered comparison with NaN is false, and `-0.0 == +0.0` is true. Generic APIs requiring `Eq`, `Ord`, or `Hash` reject floating arguments. Explicit future wrapper types may provide total or bitwise contracts without changing the primitive types.

Integer, floating-operation, conversion, and reproducibility rules are defined by the corresponding `NUM-*` rules in `CORE-V1-ABSTRACT-MACHINE.md`.

Type Safety Guarantees

1. **No null pointer dereferences**: Option type prevents null access
2. **No buffer overflows**: Array bounds checking
3. **No use-after-free**: Ownership system prevents dangling pointers
4. **No data races**: Borrowing rules prevent concurrent access violations
5. **No memory leaks**: Automatic memory management through ownership

Informative Future Directions (Non-Normative)

Lifetime Parameters

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

Trait Objects

Dynamic dispatch via `dyn Trait` is a future extension; `dyn` is a reserved keyword.

Capturing Closures

Lambda expressions that capture their environment are a future extension; the `fn(...)` function types in Core v1 are non-capturing.

C2.8 Static-Semantics Examples

These examples are parser fixtures and semantic-review cases; C2.11 supplies the positive/negative executable evidence for each granular rule.

```
type UserId = Int32;
struct User { id: UserId }
struct Recursive { next: Option<Box<Recursive>> }
// struct Invalid { next: Invalid } // rejected: infinitely sized

fn identity<T>(value: T) -> T { value }
let inferred: Int64 = identity(1);

fn inspect(value: &String) { println(value.as_str()); }
inspect(&String::from("temporary"));
// let escaped = &String::from("temporary"); // rejected: no
lifetime extension

enum Message { Quit, Count(Int32) }
fn code(message: Message) -> Int32 {
    match message {
        Message::Quit => 0,
        Message::Count(value) => value,
```

```
    }
}

const WIDTH: UInt64 = 4;
const ENABLED: Bool = WIDTH == 4;
const SETTINGS: (UInt64, Bool) = (WIDTH, if ENABLED { true } else
{ false });
```

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Semantic Analysis Specification

Overview

Semantic analysis validates that programs are meaningful according to STARK's language rules. This phase occurs after parsing and before code generation.

Analysis Phases

1. Symbol Table Construction

Build symbol tables for scopes and declarations.

Scope Rules

NAME-SCOPE-001. Modules, functions, trait/impl bodies, lexical blocks, match arms, and generic parameter lists introduce scopes. Module items and imports are visible throughout their module independent of declaration order. Function parameters and item generic parameters are visible throughout the corresponding signature/body. A `let` binding becomes visible only after its initializer and declaration complete; it is not visible in its own initializer. Pattern bindings are visible only in their selected arm body.

```
// Module scope
fn module_function() { }
const MODULE_CONST: Int32 = 42;

fn example() {
    // Function scope
```

```
    let local_var = 10;

    {
        // Block scope
        let block_var = 20;
        // block_var accessible here
        // local_var accessible here
    }
    // block_var not accessible here
    // local_var still accessible
}
```

Name Resolution Order

NAME-RESOLVE-001. Core has distinct module, type, value, and associated- item namespaces. Type context searches types, aliases, traits, and type parameters; value context searches locals, parameters, constants, functions, and constructors; a path segment naming a module searches the module namespace. Associated names are searched only after resolving their qualifying type or trait. The same spelling may coexist in different namespaces, but two declarations in one namespace and scope are duplicates.

Name resolution distinguishes *lexical* scopes (inside function bodies) from *module* scopes.

Within a function body, an unqualified name is resolved lexically: 1. Current block scope 2. Enclosing block scopes (innermost to outermost) 3. Function parameters

If not found lexically, the name is resolved at module level per the module system rules (see 07-Modules-and-Packages.md): 4. Items declared in the current module 5. Items brought into scope by `use` 6. Built-in names (prelude)

Unqualified names never implicitly search parent modules or the crate root; those require explicit `super::` or `crate::` paths.

Shadowing Rules

NAME-SHADOW-001. A local value binding may shadow a value from an outer lexical or module scope. It may not duplicate a parameter, local, or pattern binding in the same lexical scope. Item/import collisions in the same module namespace are errors rather than shadowing. Generic parameters may not duplicate another generic parameter or an item-level `Self`; a nested item introduces fresh item scopes. When a shadowing binding ends, the outer binding is visible again.

```
let x = 10;       // x: Int32
{
    let x = "hi"; // Shadows outer x, x: String
    // Inner x takes precedence
}
// Outer x visible again
```

2. Type Checking

Variable Declarations

```
let x: Int32 = 42;      // Type annotation matches literal
let y = 42;             // Type inferred as Int32
let z: String = 42;     // Error: type mismatch
```

Function Calls

```
fn add(a: Int32, b: Int32) -> Int32 { a + b }

let result = add(1, 2);    // OK: arguments match parameters
let error = add(1.0, 2);   // Error: Float64 cannot be Int32
let error2 = add(1);       // Error: wrong number of arguments
```

Assignment Compatibility

```
let mut x: Int32 = 10;
x = 20;                    // OK: same type
x = "hello";               // Error: type mismatch

let y: Int32 = 10;
y = 20;                    // Error: y is not mutable
```

3. Ownership and Borrowing Analysis

Move Semantics Validation

```
let s1 = String::from("hello");
let s2 = s1;                // s1 moved to s2
print(s1);                  // Error: use of moved value

fn take_ownership(s: String) { }
let s3 = String::from("world");
take_ownership(s3);         // s3 moved into function
print(s3);                  // Error: use of moved value
```

Borrow Checking

```
let mut s = String::from("hello");
let r1 = &s;                // Immutable borrow
let r2 = &s;                // OK: multiple immutable borrows
let r3 = &mut s;            // Error: cannot borrow mutably while
immutably borrowed

let mut s2 = String::from("world");
let r4 = &mut s2;           // Mutable borrow
let r5 = &s2;               // Error: cannot borrow immutably while
mutably borrowed
```

```
let r6 = &mut s2;         // Error: cannot have multiple mutable
borrows
```

Lifetime Validation

Returned references must obey the Core v1 reference rules defined in 03-Type-System.md ("References and Lifetimes"): a returned reference must derive from a reference parameter on every path, and callers treat it as having the shortest lifetime among the reference arguments it may derive from.

```
fn invalid_return() -> &Int32 {
    let x = 42;
    &x                    // Error: returning reference to local
variable
}

fn valid_return(x: &Int32) -> &Int32 {
    x                     // OK: returning input reference
}
```

4. Control Flow Analysis

Unreachable Code Detection

FLOW-LOOP-001. Reachability is computed structurally to a fixed point. `return`, `break`, `continue`, propagation by `?`, and expressions of type `!` do not fall through. A `loop` without a reachable `break` does not fall through; `while` and `for` are assumed able to execute zero times unless their condition/iterator is proven otherwise by constant evaluation. Statements after a non-fallthrough point are unreachable and require warning `W0005`; unreachable code is still parsed, resolved, and type-checked.

```
fn example() -> Int32 {
    return 42;
    let x = 10;           // Warning: unreachable code
}
```

Return Path Analysis

```
fn missing_return() -> Int32 {
    let x = 42;
    // Error: not all paths return a value
}

fn conditional_return(flag: Bool) -> Int32 {
    if flag {
        return 1;
    }
    // Error: missing return in else branch
}

fn valid_return(flag: Bool) -> Int32 {
```

```
    if flag {
        1
    } else {
        2
    }                     // OK: both branches return
}
```

Break/Continue Validation

```
fn invalid_break() {
    break;                // Error: break outside of loop
}

fn valid_break() {
    loop {
        break;            // OK: break inside loop
    }
}
```

5. Pattern Matching Analysis

Exhaustiveness Checking

```
enum Color { Red, Green, Blue }

fn check_color(c: Color) -> String {
    match c {
        Color::Red => "red",
        Color::Green => "green"
        // Error: non-exhaustive pattern (missing Blue)
    }
}

fn valid_match(c: Color) -> String {
    match c {
        Color::Red => "red",
        Color::Green => "green",
        Color::Blue => "blue"     // OK: exhaustive
    }
}
```

Rules: - A `match` over an enum type is exhaustive if every inhabited variant and payload space is covered, or a wildcard/binding arm covers the remainder. - Tuple, array, struct, and variant payload patterns are exhaustive only when their component pattern matrix is exhaustive. - `Bool` is exhausted by `true` and `false`; `Unit` by `()`; a zero-variant enum requires no reachable arm. Integer, float, character, string, and other open scalar domains require a wildcard/binding remainder. - If a match is not exhaustive, it is a compile-time error.

PAT-EXHAUST-001. Exhaustiveness uses a deterministic pattern-matrix specialization algorithm over normalized scrutinee types. Constructors are enum variants, tuple/array/struct shapes, `true`/`false`, unit, and scalar constant/literal tests.

The checker recursively specializes each constructor and reports a deterministic missing witness. Guards and alternative patterns do not exist in Core v1 and therefore cannot affect coverage.

Pattern Type Checking

```
let x: Int32 = 42;
match x {
    "hello" => "string",          // Error: pattern type mismatch
    42 => "number",               // OK
    _ => "other"
}
```

Pattern legality:

- wildcard matches any type and introduces nothing;
- an unresolved identifier introduces one binding; a name resolved as a constant or unit variant is a value test;
- tuple and array arity must exactly match the scrutinee;
- a struct/variant path must name the scrutinee's normalized nominal type, every named field must exist and occur once, and omitted fields remain unbound;
- literal and constant patterns must have the scrutinee type after expected- type literal inference and are restricted to primitive scalar values and unit enum variants. Equality is the compiler-known primitive/variant operation. Struct, tuple, array, payload-enum, and other nonprimitive constants are rejected as patterns even if their type implements `Eq`;
- every binding name occurs at most once in one pattern.

For an owned scrutinee, a `Copy` component gives its binding the component type by copy and a non-`Copy` component gives it that type by move. Moves from indexed places, borrowed places, or fields of a type implementing `Drop` are rejected. Regions and provenance follow the C2.8 ownership rules in `03-Type-System.md`; runtime creation and destruction order is `PAT-OWN-001`/`PAT-DROP-001`.

PAT-BIND-001 (binding mode). A `match` scrutinee is *read through a reference* when it is a dereference (`*r`), or a field/tuple-field access whose base has reference type, or such an access nested inside one. In that case a binding to a **non-`Copy`** component receives type `&C` for component type `C`, borrowing the component in place; the referent is never moved. A `Copy` component is unaffected and still binds by value, because a `Copy` read moves nothing.

For every other scrutinee — including a scrutinee that merely *has* reference type — the owned rule above applies unchanged.

Four consequences are normative rather than incidental:

- **The mode is decided per `match`, from that `match`'s own scrutinee.** It is not inherited. A binding produced by PAT-BIND-001 has reference type, and a reference-typed scrutinee is not matchable (below), so inspecting such a binding requires an explicit dereference. Nested by-reference matching is therefore written `match *inner { .. }` at each level.
- **A reference-typed scrutinee is not matched against constructor patterns.** A struct/variant path must name the scrutinee's *normalized nominal* type, and `&T` is not a nominal type, so `match r { E::V(x) => .. }` for `r: &E` is a type error. This is why the rule is stated over the *place read*, not over the scrutinee's type.

- **The rule is uniform over pattern forms**, not specific to enum payloads: a variant payload, a struct field (named or shorthand), and a tuple element all bind the same way. It has no effect outside `match`, which is Core v1's only pattern position with a scrutinee.
- **Bindings are shared even through `&mut`.** Matching through an exclusive reference still produces `&C`, never `&mut C`. In-place mutation of a matched component is consequently not expressible in Core v1. This is a deliberate floor: granting exclusive bindings is a later decision, and reserving it now keeps that decision from arriving as an implementation detail.

Compatibility. PAT-BIND-001 accepts source that earlier revisions rejected: binding a non-`Copy` component through a reference was reported as a move out of a borrow (`E0101`). No previously accepted program changes meaning — the owned rule is untouched, and the newly accepted programs had no prior behaviour to preserve. Moving such a binding's referent remains rejected, now because the binding's type is `&C` rather than `C`.

PAT-USEFUL-001. Arms are examined in source order with the same pattern-matrix algorithm. An arm is useful only if it covers at least one value not covered by earlier arms. A wholly subsumed arm is a compile-time error. Duplicate scalar constants, a constructor after a covering wildcard/binding, and structurally subsumed nested patterns are therefore rejected. Because constant-pattern equality never invokes user code, usefulness is decidable and pattern tests have no user-defined side effects or traps.

6. Mutability Analysis

Mutable Access Validation

Assignment to an initialized immutable variable is an error. (Exception: the single initializing assignment of a deferred-initialization `let` — see Initialization Analysis.)

```
let x = 42;
x = 43;                    // Error: x is not mutable

let mut y = 42;
y = 43;                    // OK: y is mutable

struct Point { x: Int32, y: Int32 }
let p = Point { x: 1, y: 2 };
p.x = 10;                  // Error: p is not mutable

let mut p2 = Point { x: 1, y: 2 };
p2.x = 10;                 // OK: p2 is mutable
```

7. Initialization Analysis

Use Before Initialization

```
let x: Int32;
print(x.fmt());            // Error: use of uninitialized variable
```

```
let y: Int32 = if true { 42 } else { 24 };
print(y.fmt());           // OK: y is initialized in all branches

let z: Int32;
if condition {
    z = 42;
}
print(z.fmt());           // Error: z might not be initialized
```

Rules: - `let name: Type;` declares a variable without initializing it. - A variable must be definitely assigned before any read. - All control-flow paths must assign before use. - **Deferred initialization of immutable variables**: an *uninitialized* immutable `let` may be assigned exactly once on each control-flow path; this first assignment is initialization, not mutation. Any assignment to an already-initialized immutable variable is an error (see Mutability Analysis).

Double Initialization

```
let mut x: Int32 = 42;
x = 43;                   // OK: reassignment
let x = 44;               // Error: redeclaration in same scope
```

8. Array Bounds Analysis

Static Bounds Checking

FLOW-BOUNDS-001. Indexing an array by a compile-time constant is rejected when the value is outside `0 <= index < N`. A constant range is rejected when either endpoint is outside `0..=N`, when start exceeds end, or when an inclusive end would require an element at `N`. The same proof is required for constant `Vec`/slice lengths only when their length is itself statically known. Failure to prove an error preserves the abstract-machine runtime check; it is never permission to omit that check.

```
let arr: [Int32; 3] = [1, 2, 3];
let x = arr[2];           // OK: index in bounds
let y = arr[5];           // Error: index out of bounds (if
determinable)
```

Dynamic Bounds Checking

```
let arr: [Int32; 3] = [1, 2, 3];
let idx = get_index();
let x = arr[idx];         // Runtime bounds check required
```

9. Error Propagation Analysis

Question Mark Operator

```
fn might_fail() -> Result<Int32, String> { Ok(42) }
```

```
fn caller1() -> Result<Int32, String> {
    let x = might_fail()?;    // OK: compatible error types
    Ok(x * 2)
}

fn caller2() -> Int32 {
    let x = might_fail()?;    // Error: function doesn't return
Result
    x * 2
}
```

Rules: - `expr?` propagates `Err` or `None` to the nearest enclosing function returning `Result<_, E>` or `Option<_>`. - The enclosing function return type must be compatible with the propagated type.

Runtime Error Semantics (Core v1)

Static analysis must identify operations whose normative runtime rule can trap and preserve the source location of that operation. Trap categories, abort behavior, and the absence of unwinding are defined solely by `CORE-V1-ABSTRACT-MACHINE.md` (`TRAP-CATEGORY-001` and `DROP-ABORT-001`). Numeric and process-specific classifications are completed by C2.9.

10. Trait Constraint Checking

Trait Bounds Validation

```
trait Display {
    fn fmt(&self) -> String;
}

fn print_it<T: Display>(item: T) {
    print(item.fmt());        // OK: T implements Display
}

struct Point { x: Int32, y: Int32 }

print_it(Point { x: 1, y: 2 });  // Error: Point doesn't implement
Display
```

Error Reporting

Error Message Format

```
Error: [ERROR_CODE] [BRIEF_DESCRIPTION]
  --> file.stark:line:column
   |
line | source code line
   | ^^^^^ specific location
```

```
  |
  = help: detailed explanation
  = note: additional information
```

Error Categories

Type Errors (E0001-E0099)

- E0001: Type mismatch
- E0002: Unknown type
- E0003: Type annotation required
- E0004: Cannot infer type
- E0005: Wrong number of arguments
- E0006: `?` operator in a function that does not return `Result` or `Option`
- E0007: Index out of bounds (determinable at compile time)
- E0008: Integer literal out of range for its type (suffixed literal exceeds its suffix's representable range, or an unsuffixed literal exceeds `Int64`)
- E0009: Array repeat count (`[value; count]`) is not a compile-time constant expression

Ownership Errors (E0100-E0199)

- E0100: Use of moved value
- E0101: Borrow check failed
- E0102: Lifetime violation
- E0103: Dangling reference
- E0104: By-value iteration over a fixed-length array with a non-`Copy` element type is not yet supported (deferred feature — consuming each element moves it out of a place named by a runtime loop index; iterate over a borrow instead. `Copy` arrays iterate normally.)
- E0105: Iteration form not supported by this implementation (deferred feature). Covers by-value iteration over `Vec<T>` and the `Iterator` combinator methods (`map`, `filter`, `count`, `collect`, `fold`, `reduce`, `any`, `all`, `find`). Iterating a borrow — either `for x in &v` or `for x in v.iter()`, which are the same iteration and both yield `&T` — and driving an iterator with a `for` loop are supported. An implementation that supports these forms does not raise E0105; one that does not MUST raise it during semantic analysis rather than accepting the program and failing later, so that acceptance and executability agree.

Name Resolution Errors (E0200-E0299)

- E0200: Undefined variable
- E0201: Undefined function
- E0202: Undefined type
- E0203: Ambiguous name
- E0204: Duplicate definition in the same scope
- E0205: Unresolved import
- E0206: `super` used from a root module
- E0207: Private item is not visible from the use site
- E0208: Module file missing, unreadable, or ambiguous
- E0209: Public API exposes a type that is not publicly nameable
- E0210: Extension-only syntax used without enabling its extension

- E0211: Invalid extension declaration or argument
- E0212: Extension type, shape, device, or range mismatch
- E0213: Missing, malformed, or invalid extension generic argument
- E0214: Invalid executable entrypoint selection or signature
- E0215: Constant expression is outside the Core subset or failed compile-time evaluation
- E0216: Recursive transparent type-alias cycle
- E0217: Unsized value position or infinite-size direct value cycle

Control Flow Errors (E0300-E0399)

- E0301: Missing return value
- E0302: Break outside loop
- E0303: Non-exhaustive match
- E0304: Method receiver has a type on which methods cannot be called
- E0305: Constant pattern is not a compiler-known primitive scalar

(E0300 is intentionally unassigned: unreachable code is warning W0005, not an error — see “Unreachable Code Detection” above.)

Mutability and Initialization Errors (E0400-E0499)

- E0400: Assignment to immutable binding
- E0401: Use of possibly-uninitialized variable

Trait Errors (E0500-E0599)

- E0500: Trait not implemented

Warning Categories

Unused Items (W0001-W0099)

- W0001: Unused variable
- W0002: Unused function
- W0003: Unused import
- W0004: Dead code
- W0005: Unreachable code
- W0006: Unreachable match arm

Style Warnings (W0100-W0199)

- W0100: Non-snake_case variable
- W0101: Non-PascalCase type
- W0102: Missing documentation

Conformance

A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Core v1 Abstract Machine

Status and authority

This document is normative for Core v1. It is the sole authority for runtime evaluation, values and places, ownership transfer, temporary lifetime, destruction, references, and language traps. `03-Type-System.md` defines static legality, `04-Semantic-Analysis.md` defines required analyses and rejection categories, and `05-Memory-Model.md` summarizes safety guarantees. None of those documents defines a competing execution model.

The abstract machine does not prescribe an interpreter frame layout, stack or heap placement, pointer representation, object layout, garbage collector, MIR shape, backend ABI, or optimizer. A conforming implementation may represent the concepts below in any way that preserves all specified behavior.

Terms and machine state

Values, objects, storage, and ownership

AM-VALUE-001. A *value* is a typed Core datum: a scalar, function reference, reference, aggregate, enum value, or value of an implementation-provided Core library type.

AM-OBJECT-001. An *object* is a value together with its identity and lifetime. Object identity is abstract. It is not a numeric address and does not change merely because ownership of the object moves between places.

AM-STORAGE-001. A *storage location* is an abstract slot capable of holding one object or being uninitialized. An *allocation* is one or more storage locations whose physical placement is not observable unless another Core rule explicitly exposes it.

AM-OWNER-001. Every live non-`Copy` object has exactly one owner. An owner may be a local, temporary, aggregate field or element, active enum payload, function parameter, return-transfer slot, iterator, collection, or another library value whose contract owns the object.

AM-LOCAL-001. A *local binding* names a storage location within a lexical scope. A parameter and method receiver are locals whose initialization occurs on function entry. A local is *initialized* when its location contains a live object and *moved-from* when ownership has been transferred out without replacement.

AM-TEMP-001. A *temporary* is an unnamed owner created while evaluating a full expression. Temporaries have the same exactly-once destruction obligations as named locals.

Places and projections

EXEC-PLACE-001. A *place* designates a storage location or a sublocation of an object. Core places are:

- an initialized local, parameter, or receiver;
- a field or tuple-field projection from a place;
- an array, slice, vector, map-entry, or other indexing projection whose library contract defines a place;
- a range projection denoting a slice place;
- dereference of a valid reference.

A projection is part of the place identity; resolving a place does not read or move its stored value. Field and tuple projections resolve their base first. Index projections resolve the base place, then evaluate the index. Bounds or key failure is a language trap.

An expression used where a place is required must be one of the place forms above, except that borrowing an rvalue may first materialize that rvalue in a temporary place.

Evaluation model

Results and full expressions

Evaluation of an expression produces exactly one of:

- a value;
- a place when the surrounding context requires a place;
- `return`, carrying a value;
- `break`, optionally carrying a value;
- `continue`;
- propagation from `?`, carrying `None` or `Err`;
- a language trap.

A *full expression* is the outermost expression of an initializer, expression statement, `return` operand, `break` operand, condition, match scrutinee, loop iterable, aggregate initializer, assignment operand, or block tail. Operands, call arguments, call receivers, callees, and aggregate fields/elements are subexpressions of that enclosing full expression, not independent temporary-destruction boundaries.

EXEC-ONCE-001. Every evaluated expression and subexpression is evaluated exactly once. Static dispatch, method lookup, place resolution, auto-borrowing, and auto-dereferencing must not re-evaluate source expressions.

EXEC-DISPATCH-001. Execution invokes exactly the function, inherent method, trait method, default method, associated item, or compiler-recognized language hook selected by the normative static rules. A runtime may not substitute source-order lookup, same-named inherent lookup for a trait-qualified protocol, structural host equality/ordering, or string-name dispatch with different semantics. Dispatch selection itself has no Core-visible side effect.

Value and place contexts

MOVE-READ-001. Reading a place in a value context:

- copies the value and leaves the place initialized when its type is `Copy`;
- otherwise transfers ownership from the place and leaves that place moved-from;
- does not move when the construct explicitly borrows the place or a trait/operator rule specifies a reference argument.

The static rules reject any read that would move from a prohibited place, conflict with a live borrow, or read an uninitialized or moved-from place. Runtime execution of a well-typed program therefore never relies on recovering from such a read.

Evaluation order

EXEC-EVAL-001. Unless a row below states otherwise, subexpressions evaluate left to right in source order, and each completes—including its side effects, ownership transfers, normal destruction, control transfer, or trap—before the next begins.

Expression form	Normative order and result
Literal, function item, constant, unit	Produce the corresponding value. Constant evaluation is defined by `03-Type-System.md`.
Local/path in value context	Resolve the named place, then apply `MOVE-READ-001`.
Grouping	Evaluate the grouped expression once.
Field or tuple field	Resolve the base place, append the projection, then read only if a value is required.
Integer index	Resolve the base place, evaluate the index, check bounds, append the projection, then read only if required.
Range index	Resolve the base place, evaluate the range bounds left to right, validate them, and produce a slice place/view.
Borrow &/&`mut`	Resolve the operand place, materializing an rvalue temporary if

Expression form	Normative order and result
	permitted, then create a reference without reading the referent.
Dereference	Evaluate the reference, validate it, and produce its referent place; read only if required.
Other unary operation or cast	Evaluate the operand, then apply the operation. Numeric outcomes are completed by C2.9.
Non-short-circuit binary operation	Evaluate the left operand, then the right, then apply the operation.
`a && b`	Evaluate `a`; evaluate `b` only when `a` is `true`.
`a || b`	Evaluate `a`; evaluate `b` only when `a` is `false`.
Range construction	Evaluate the lower bound, then upper bound, then construct the range.
Free or associated call	Select the statically named function item without runtime callee evaluation; evaluate arguments left to right; transfer them to parameters; execute the body.
Function-valued call	Evaluate the callee expression exactly once and verify that its result is callable; evaluate arguments left to right; transfer them to parameters; invoke the selected function. A callee trap prevents every argument; an argument trap prevents later arguments and body execution.
Method call	Evaluate and resolve the receiver exactly once; evaluate arguments left to right; transfer/borrow the receiver and arguments; execute the selected body.
Tuple, array, struct, enum payload	Evaluate fields/elements left to right in written order, transferring each completed value into the partial aggregate.
Array repeat `[value; count]`	Evaluate `value` once; obtain the compile-time `count`; copy the value `count` times. The static rules require `Copy`.
`if`	Evaluate the condition, then only the selected branch.
`match`	Evaluate the scrutinee exactly once; test arms in source order; evaluate only the first matching arm.
Block	

Expression form	Normative order and result
	Evaluate statements in order, then the optional tail expression; the block result is the tail value or `Unit`.
`loop`	Repeatedly evaluate the body until control transfers.
`while`	Evaluate the condition before each iteration and the body only when it is `true`.
`for`	Evaluate the iterable once, obtain its iterator, repeatedly obtain one next value and execute one iteration.
`?`	Evaluate the operand once; unwrap `Some`/`Ok`, or transfer `None`/`Err` toward the enclosing function.
`return`, `break`	Evaluate the optional operand before beginning the control transfer.
`continue`	Begin the control transfer without an operand.
Assignment	Follow `EXEC-ASSIGN-001`; it is the named right-before-left exception.

An expression form not admitted by the Core grammar has no Core execution semantics. A legal Core expression form omitted from this table is a specification defect, not permission for implementation-defined evaluation.

A temporary function value produced by a function-valued callee expression remains live through argument evaluation and the invoked body. Unless ownership transfers elsewhere, it is destroyed with the other temporaries at the end of the enclosing full expression.

EXEC-FOR-001. Every value accepted by the static `for`-loop iterator rules is executed through its selected iterator protocol. The runtime may not narrow this to a privileged list of container representations. The iterable expression evaluates once, each successful step yields one iteration value, and no later step occurs after loop control exits.

Assignment and replacement

EXEC-ASSIGN-001. Simple assignment `destination = source` executes in this exact order:

1. evaluate `source` completely and retain ownership of its result;
2. resolve `destination` as a place, including evaluation and validation of its projections;
3. detach the old destination object if the place is initialized;
4. write the new object and make the destination initialized;
5. destroy the detached old object.

The new value is installed before user-observable destruction of the old value begins. If the old value's destructor traps, execution aborts with the new value already installed, although subsequent program observation is impossible because Core has no trap recovery.

Compound assignment evaluates the right operand first, resolves the destination second, reads the old destination value exactly once, applies the corresponding operation, and replaces the old value using steps 3–5 above.

If source evaluation traps, the destination is not resolved or modified. If destination resolution traps after the source has produced an owned object, the program aborts; the source object and all other live values are not destructed because `DROP-ABORT-001` forbids unwinding.

```
let mut values = [0, 0];
values[next_index()] = make_value(); // make_value() completes
before next_index().

let mut values = [0];
values[1] = String::from("owned"); // RHS is produced; index
failure then aborts without Drop.
```

Aggregate construction

EXEC-AGG-001. Aggregate fields and elements become owned by a partial aggregate as each initializer completes. Declaration order determines layout-independent field identity; written initializer order determines evaluation order. Duplicate, unknown, or missing required fields are static errors and do not execute.

Tuple and positional enum payload order is index order. Named struct and enum payload initializers evaluate in written order even when that differs from declaration order. The completed object's later field-destruction order is declaration order reversed, not initializer order reversed.

EXEC-AGG-002. If a later initializer performs normal early control transfer, already completed fields are destroyed in reverse completion order before the transfer continues. If a later initializer traps, construction aborts and no completed field is destructed, because a trap never unwinds.

Core v1 has no struct-update syntax. If a later edition adds it, that edition must define its evaluation, transfer, and failure sequence rather than inheriting one implicitly.

```
let value = (Loud::new("completed"), fail()); // A trap in fail()
does not unwind completed.
```

Normal control transfer and partial evaluation

EXEC-CFLOW-001. `return`, `break`, `continue`, and `?` propagation are normal control transfers, not traps. Before a transfer leaves a scope, all live owners in that scope that are not carried by the transfer are destroyed using the normal order. The carried value is moved out before cleanup and is not destroyed by the exited scope.

For nested scopes, cleanup proceeds from the innermost exited scope outward. `break` destroys the current iteration and loop-body state before producing the loop result. `continue` destroys the current iteration and begins the next. `?` destroys scopes between the operator and the function boundary before transferring `None`/`Err` to the caller.

Moves, partial moves, and reinitialization

OWN-MOVE-001. Passing a non-`Copy` value to a by-value parameter, storing it in an aggregate, returning it, breaking with it, propagating it, or binding it by value transfers ownership. Passing or returning a `Copy` value copies it.

OWN-PARTIAL-001. Moving a field from an aggregate transfers only that field and marks that field moved-from. Remaining initialized fields remain individually usable and destructible. The whole aggregate cannot be read, moved, or destructured as fully initialized until every moved field is reinitialized. Moving a field from a type that implements `Drop` is prohibited, because its destructor requires the complete value.

Moving out of an indexed place is prohibited unless a library operation explicitly owns and removes that element (for example `Vec::remove` or `pop`). This keeps shifting/container invariants inside the owning operation.

OWN-REINIT-001. Assignment to a moved-from mutable place reinitializes that place. Assigning all moved fields reconstitutes a fully initialized aggregate. Reinitialization does not destroy the absent old value.

```
let mut text = String::from("first");
let moved = text;
text = String::from("second"); // text is initialized again; no
old text remains to drop.
```

Pattern execution

This section defines the C2.7 ownership/destruction mechanics for the pattern forms present in Core v1. C2.8 remains authoritative for pattern typing, exhaustiveness, usefulness, and any additional static restrictions.

PAT-OWN-001. A `match` scrutinee in value context is evaluated once. A non-`Copy` place scrutinee moves into a hidden scrutinee owner; a `Copy` scrutinee is copied. Pattern traversal follows written source order, recursively left to right. Tuple, array, and positional payload patterns therefore use index order; named struct and enum patterns use the written field order, even when it differs from declaration order.

For each binding reached by that traversal:

- a matched `Copy` component is copied into the binding and remains initialized in the hidden scrutinee;
- a matched non-`Copy` component moves into the binding and becomes moved-from in the hidden scrutinee;

- a reference scrutinee produces the reference projection required by the static pattern rules and does not move or copy the referent.

The wildcard _ creates no binding and does not suppress destruction.

PAT-DROP-001. The selected arm's bindings are arm-local owners. On every normal exit from the arm, binding locals are destroyed in reverse creation order after any arm result has moved out. Then every still-owned, unbound component of the hidden scrutinee—including copied source components and wildcarded or omitted fields—is destroyed exactly once in reverse declaration/index order. For a named pattern, binding creation order is written pattern order, while residual field destruction is reverse source declaration order. A trap in pattern testing or the arm aborts without either cleanup.

Failed arm tests create no user-visible bindings and transfer no ownership out of the hidden scrutinee. Core v1 has no match guards or alternative (`|`) patterns.

```
struct Loud { label: String }
impl Drop for Loud { fn drop(&mut self)
{ println(self.label.as_str()); } }
let pair = (Loud { label: String::from("bound") },
            Loud { label: String::from("wildcard") });
match pair { (kept, _) => println(kept.label.as_str()), }
```

References and borrow-carrying values

Identity and derivation

REF-IDENTITY-001. A reference is a value containing:

- the identity of a live referent object;
- a projection path and, for a slice, validated bounds;
- shared or exclusive access capability;
- the static validity interval established by the type/borrow rules.

This is an abstract identity, not a promised address or pointer width. Two references alias when they designate overlapping storage of the same object.

Creating a reference does not copy or move the referent. Moving the reference value preserves its designation. Moving the referent's owner while a conflicting reference is live is rejected statically. When an ownership move is legal, object identity is preserved even if physical storage changes.

Projection, auto-borrow, and auto-dereference

REF-PROJECT-001. Dereference yields the designated place. Field, tuple, index, and slice projections through a reference extend its projection path and retain the referent's identity. Reads and writes through the resulting place observe the current object, never a snapshot.

Method auto-borrow creates a reference to the already evaluated receiver place. Auto-dereference follows references without re-evaluating the receiver. An exclusive receiver requires a mutable place and preserves write-through behavior.

Returned and receiver-derived references

REF-RETURN-001. A returned reference must designate an object whose validity, under the static shortest-input-lifetime rules, extends into the caller. Transfer across a call boundary preserves referent identity and projection. A reference derived from `&self`, `&mut self`, or another reference parameter designates the caller's object—not a parameter copy or callee-local storage—and remains valid after the callee's activation ends for the statically permitted interval.

```
struct Cell { value: Int32 }
impl Cell {
    fn value_ref(&self) -> &Int32 { &self.value }
}
let cell = Cell { value: 42 };
let reference = cell.value_ref();
```

Slice references

REF-SLICE-001. Borrowing a range-indexed place creates a view of the original base object. The view records validated start/end bounds and aliases those elements. Reads observe later legal writes; writes through an exclusive slice reference update the original object. Creating or returning a slice never clones its elements merely to satisfy reference semantics.

```
struct Numbers { items: Vec<Int32> }
impl Numbers {
    fn middle(&self) -> &[Int32] { &self.items[1..3] }
}
let mut items = Vec::new();
items.push(10);
items.push(20);
items.push(30);
let numbers = Numbers { items };
let middle = numbers.middle();
```

Borrow-carrying values

REF-CARRY-001. A generic aggregate or library value that contains references carries their referent identities, projections, capabilities, and validity requirements unchanged through moves, calls, returns, and pattern binding. Moving the carrier does not retarget its references. Destroying a carrier destroys owned non-reference components but does not destroy referenced objects.

Destruction

Exactly-once rule

DROP-EXACT-001. Every initialized owned object that reaches a normal destruction point is destroyed exactly once. A moved-from or never-initialized place is not destroyed. Moving a value transfers its destruction obligation to the new owner. `Copy` types cannot implement `Drop`.

For a type implementing `Drop`, its `Drop::drop(&mut self)` body runs first while all non-moved fields remain readable. After it returns normally, owned fields are recursively destroyed. Calling `Drop::drop` directly is prohibited; the `drop(value)` function consumes its argument and performs this sequence.

If a destructor traps, execution aborts immediately; remaining fields, siblings, locals, and outer scopes are not destructed.

Deterministic order

DROP-ORDER-001. On normal execution:

- expression-statement results are destroyed at the statement boundary;
- temporaries are destroyed in reverse creation order at the end of their full expression;
- block locals are destroyed in reverse declaration order;
- a return/break/propagated result moves out before local cleanup;
- function body locals are destroyed as their scopes exit, then parameters in reverse parameter order; a receiver is the first parameter for this purpose;
- tuple and array elements are destroyed in reverse index order;
- struct and named enum-variant fields are destroyed in reverse source declaration order;
- positional enum payloads are destroyed in reverse index order;
- only the active enum payload is destroyed;
- `Option`, `Result`, `Box`, and similar single-payload owners destroy their active payload;
- partially moved aggregates destroy only fields that remain initialized.

The order is defined in terms of source declarations and logical ownership, never a map order, address order, backend layout, or optimizer choice.

Temporaries

EXEC-TEMP-001. A temporary that is not transferred into another owner lives until the end of its enclosing full expression and is then destroyed in reverse creation order. An rvalue materialized so it can be borrowed as a call argument remains live through all later argument evaluation and through completion of the call. An rvalue materialized as a borrowed method receiver likewise remains live through argument evaluation and method completion. A temporary function value used as a callee follows the same call-completion rule.

The abstract machine does not, by itself, extend an rvalue temporary merely because its reference is stored in a local or returned. C2.8's static borrow/escape rules decide whether such a reference is legal and must reject any accepted reference whose validity would outlast the referent storage assigned by those rules. A temporary moved into a local, aggregate, argument, return value, or other owner ceases to be a temporary and its destruction obligation transfers.

A block tail value is moved or copied out before the block's locals and remaining temporaries are destroyed. Temporaries created while evaluating a condition, match scrutinee, loop iterable, argument, or aggregate initializer follow the more specific ownership rules for that construct.

Loops and iterators

DROP-LOOP-001. A `for` iteration binding owns the yielded value for exactly one iteration. After the body exits normally, by `continue`, by `break`, by `return`, or by `?`, the binding is destroyed unless its value was moved out. Body-local destruction precedes iteration-binding destruction.

The iterator owns its unconsumed state. On normal loop completion or normal early transfer, the iterator is destroyed and therefore destroys all remaining owned elements according to its library contract. A trap aborts without this cleanup.

```
for item in values {
    if skip(item) { continue; }
    if stop(item) { break; }
    consume(item);
}
```

Collections and ownership-discarding operations

DROP-COLLECTION-001. A collection owns its stored elements and entries. Destroying or clearing a sequence/set destroys elements in reverse defined iteration order. Destroying or clearing a map destroys entries in reverse defined iteration order, destroying each value before its key. Operations with component-wise return contracts transfer only the returned components and destroy the remaining removed or rejected components:

- `HashMap::insert` with a new key transfers the supplied key and value into the map and returns `None`;
- `HashMap::insert` with an equal existing key retains the stored key and its iteration position, installs the supplied new value, transfers the old stored value into `Some(old_value)`, and destroys the supplied duplicate key before returning; the returned `Option<V>` owns the old value;
- `HashMap::remove(&key)` leaves the lookup reference and referent caller-owned, removes the stored entry, destroys the stored key, and transfers the stored value into `Some(value)`; absence returns `None` and destroys nothing;
- `HashSet::insert` with a duplicate destroys the rejected supplied value and returns `false`; a new value transfers into the set and returns `true`;
- `HashSet::remove(&value)` leaves the lookup reference and referent caller-owned, destroys the removed stored value, and returns `true`; absence returns `false`.

For any other removal or replacement API, a component named in the return value transfers to that return owner; an owned component not returned is destroyed by the operation. Ignoring a returned owner does not move destruction into the collection operation: the expression- statement result is a temporary and is destroyed at its normal temporary boundary.

```
let mut map: HashMap<String, Loud> = HashMap::new();
map.insert(String::from("key"), Loud::new("old"));
let replaced = map.insert(String::from("key"), Loud::new("new"));
match replaced {
    Some(old_value) => consume(old_value),
    None => {},
}
```

Focused ordering examples

The following examples isolate ordering rules that are easy for a backend to implement accidentally.

```
fn choose_function() -> fn(Int32) -> Int32 { selected }
let result = choose_function()(make_argument());
```

Here `choose_function` completes before `make_argument`; if it traps, `make_argument` does not run.

```
struct Record { count: Int32, label: String }
let record = Record { count: 3, label: String::from("owned") };
match record {
    Record { label: text, count: n } => consume(text, n),
}
```

The written pattern creates `text` first by moving `label`, then creates `n` by copying `count`.

```
struct Inner { left: Loud, right: Loud }
struct Outer { first: Loud, second: Inner }
let value = make_outer();
match value {
    Outer {
        second: Inner { right: b, left: a },
        first: c,
    } => consume(b, a, c),
}
```

Binding creation order is `b`, `a`, `c`; normal binding destruction order is `c`, `a`, `b`.

Trap termination

DROP-ABORT-001. A language trap is aborting. It performs no stack unwinding, scope cleanup, temporary cleanup, partial-aggregate cleanup, collection cleanup, or user `Drop` calls for live objects. Side effects and destruction completed before the trap remain observable. Core v1 has no catch or recovery mechanism.

`panic(message)` emits its specified panic diagnostic/message and then traps. The precise process exit-code mapping is owned by C2.9; the abstract exit category is `trap`.

```
struct Loud { label: String }
impl Drop for Loud { fn drop(&mut self)
{ println(self.label.as_str()); } }
let live = Loud { label: String::from("not dropped") };
panic("abort");
```

Numeric operations and conversions

NUM-INT-ARITH-001. Signed integers are fixed-width two's-complement values and unsigned integers are fixed-width binary values of the width in their type name. Integer addition, subtraction, multiplication, exponentiation, and unary negation compute the mathematical result and trap when it is not representable in the result type. This checked behavior is identical in all build modes and targets. Negating an unsigned value is ill-typed; negating the minimum signed value traps. Integer exponentiation requires a nonnegative exponent and traps otherwise; each intermediate multiply is checked.

NUM-INT-DIV-001. Integer division by zero and remainder by zero trap. Signed division truncates toward zero; the remainder satisfies `a == (a / b) * b + (a % b)` and has the sign of `a` or is zero. Dividing the minimum signed value by `-1`, and taking its remainder by `-1`, trap because the intermediate quotient is not representable. Unsigned division and remainder use ordinary Euclidean nonnegative quotient/remainder.

NUM-SHIFT-001. A shift count may have any integer type, but its mathematical value must be nonnegative and strictly less than the bit width of the left operand; otherwise the operation traps. Left shift traps when the mathematical result is not representable. Unsigned right shift fills with zero; signed right shift is arithmetic and rounds toward negative infinity. No shift count is masked or reduced modulo the width.

NUM-CAST-001. Numeric conversion is checked as follows:

- integer to integer preserves the mathematical value and traps if the target cannot represent it;
- `Float32` to `Float64` is exact; `Float64` to `Float32` rounds once using round-to-nearest, ties-to-even and may produce signed infinity;
- integer to float rounds once using round-to-nearest, ties-to-even;
- finite float to integer first truncates toward zero and then traps unless the result is representable; NaN and either infinity always trap.

Conversion preserves a floating zero's sign and infinity's sign. A statically evaluated failing conversion is a compile-time error instead of a runtime trap.

NUM-FLOAT-OP-001. Each primitive floating `+`, `-`, `*`, `/`, unary `-`, and comparison uses IEEE 754 binary32/binary64 with round-to-nearest, ties-to-even. Floating division by zero does not trap: it produces the IEEE infinity or NaN result. Floating `%` is the correctly rounded value of `x - trunc(x / y) * y` using the exact mathematical quotient, with the sign of a nonzero result matching `x`; zero divisor, infinite dividend, or NaN operand produces NaN. NaN propagates as a quiet NaN; operations that create a NaN produce the canonical quiet NaN with sign zero and all payload bits other than the quiet bit zero. Negation flips the sign bit, including for zero and NaN. Implementations may not reassociate operations, contract multiply-add, flush subnormals, or use a different rounding mode.

Core v1 has no floating `**` operator. Floating exponentiation is the `std::math::pow(Float64, Float64)` library operation governed by `STD-MATH-001`; use of `**` with either floating operand is a type error.

For the same declared float type, inputs, and sequence of primitive operations/casts, the result bits are backend- and target-independent under `NUM-FLOAT-REPRO-001`. Decimal literals are converted directly to the destination format using round-to-nearest, ties-to-even, independent of host parsing. Transcendental and other standard-library math functions follow `STD-MATH-001`; they are not primitive operations and need not be bit-identical across targets.

Trap categories

TRAP-CATEGORY-001. A *language trap* is a failure explicitly required by a normative Core rule, including explicit `panic`, bounds failure, invalid range boundary, division by zero, checked arithmetic failure, and failing checked conversion where the owning numeric/text rule requires a trap. It records a stable category and the source location of the operation that failed.

Allocation exhaustion, stack exhaustion, recursion/call-depth exhaustion, unavailable host services, host I/O failure outside an API's `Result`, target failure, and OS termination are `host/process failure`, not language traps. Ordinary file/stream failures handled by the standard-library contract produce `Result` and do not terminate execution. Compiler limits reject compilation with a classified diagnostic. An implementation must never report an internal host panic as a specified STARK trap.

Observable execution and differential comparison

OBS-COMPARE-001. For a fixed program, inputs, target contract, enabled extensions, artifact set, and external environment, execution backends are semantically equal only when all applicable observations match:

- stdout bytes in order;
- stderr bytes in order;
- abstract exit category (`normal(status)`, `language trap`, `host/process failure`);
- returned Core value for a harnessed function, compared by its Core value semantics;
- language-trap category;
- language-trap source identity and span;
- user-observable `Drop` side effects and their order;
- artifact verification result and artifact identity when verification is part of the execution request.

Wall-clock time, physical addresses, allocation strategy, stack depth, object layout, generated code bytes, and non-normative diagnostic prose are not observations. Numeric target variation, process status numbers, environmental I/O, and resource failures are compared only after C2.9 classifies them.

Conformance

A conforming Core v1 implementation must implement every rule in this document. Optimization may omit work only when it cannot change an observation in `OBS-COMPARE-001`, including destructor effects and trap order. The interpreter, future MIR interpreter, and native backend are implementations of this abstract machine; none is an independent semantic authority.

STARK Core v1 Memory-Safety Model

Status and authority

This chapter states Core v1 memory-safety guarantees. Static ownership, borrowing, lifetime, `Copy`, and `Drop` legality is defined by `03-Type-System.md` and checked as required by `04-Semantic-Analysis.md`. Runtime values, places, moves, references, temporary lifetime, destruction order, and traps are defined solely by `CORE-V1-ABSTRACT-MACHINE.md`.

This chapter does not define physical layout. Core v1 makes no general promise that a value is stack-allocated or heap-allocated, that a reference is a machine pointer, that a slice is two machine words, or that an enum uses a particular tag. Observable layout and target contracts are owned by C2.9.

Safety invariants

For every well-typed Core v1 program that has not terminated through a language trap or an external failure:

1. every live non-`Copy` object has exactly one owner;
2. a moved-from or uninitialized place cannot be read;
3. a live shared reference permits reads but not mutation through that reference;
4. a live exclusive reference excludes conflicting shared or exclusive access;
5. a reference cannot be used outside its statically valid interval;
6. dereference and projection preserve referent identity and cannot silently produce a disconnected snapshot;
7. bounds-checked places cannot access storage outside their designated object or slice;
8. every owned object that reaches normal cleanup is destroyed exactly once;
9. a value is `Copy` only when its complete type satisfies the `Copy` legality rules;
10. normal control transfer preserves ownership and cleanup obligations.

These are language guarantees, not implementation strategies. A compiler may erase borrow metadata and omit redundant runtime checks after proving that the same observable behavior and safety invariants remain.

Ownership and moves

Ownership transfer does not duplicate the transferred object. Passing or returning a non-`Copy` value, storing it into another owner, binding it by value, or removing it through an owning collection operation transfers the destruction obligation with the object.

Partial-move legality, prohibited indexed moves, and reinitialization checks are defined by `03-Type-System.md`. Their runtime effect is defined by abstract-machine rules `OWN-PARTIAL-001`, `OWN-REINIT-001`, and `DROP-EXACT-001`.

Borrowing and validity

Core v1 has shared and exclusive references. The static rules conservatively determine their validity without written lifetime parameters. Returned references must derive from permitted reference inputs, and borrow-carrying generic values carry the same validity constraints as their contained references.

Reference identity, projection, method receivers, returned references, slice views, moves of reference carriers, and caller/callee boundaries are defined by `REF-IDENTITY-001` through `REF-CARRY-001` in `CORE-V1-ABSTRACT-MACHINE.md`.

Destruction safety

The type system prohibits a type from implementing both `Copy` and `Drop`, prohibits explicit calls to `Drop::drop`, and prohibits partial field moves from a type whose destructor requires the complete value.

The abstract machine defines all destruction points and ordering, including locals, temporaries, parameters, fields, partial aggregates, loop bindings, iterators, collections, explicit `drop(value)`, replacement assignment, normal early transfer, and aborting traps. No physical container order or object layout may substitute for the specified logical order.

Traps and external failures

A specified language trap preserves memory safety by terminating execution; Core v1 does not unwind or run remaining destructors. Side effects completed before the trap remain observable. `TRAP-CATEGORY-001` distinguishes language traps from host panics and external failures.

Allocation exhaustion, stack exhaustion, OS termination, host I/O failure, and target limits are not memory-safety loopholes and are not automatically STARK traps. C2.9 classifies their portable guarantees.

Informative future directions (non-normative)

Written lifetime parameters, reference fields, reference counting, interior mutability, concurrency, and unsafe/native memory access are not Core v1 features. A future edition must preserve the invariants above or explicitly version its compatibility boundary.

Conformance

A conforming implementation must enforce the static rules and preserve these safety invariants while implementing the abstract machine. Deviations and extensions must be documented and tested; an implementation representation is never evidence that a different language rule applies.

STARK Standard Library Specification

Overview

The STARK standard library provides essential types, functions, and modules for core programming tasks. This specification defines the minimal standard library for the initial implementation.

Notation

The code blocks in this document are **API notation**, not compilable STARK source: function signatures are listed without bodies, and `impl` blocks mix signatures with prose comments. The *signatures and behaviors* are normative; the listing style is not. (The Core v1 grammar has no body-less function form outside `trait` blocks.)

Implementation-Provided Types

Several standard library types (`Box<T>`, `Vec<T>`, `String`, `HashMap<K, V>`) manage raw memory internally. Raw pointers and `unsafe` code are NOT part of the Core v1 language surface; these types are *implementation-provided*: their public APIs are normative, their internal representation is not expressible in Core v1 source and is implementation-defined. Struct bodies shown for these types in this document are illustrative only.

Iterator and View Types

The iterator types returned by the APIs below (VecIter<T>, KeysIter<K>, ValuesIter<V>, Iter<K, V>, Iter<T>, CharsIter, SplitIter, MapIter<I, U>, FilterIter<I>) are implementation-provided opaque structs. Each implements Iterator with the obvious Item:

Type	Produced by	Item
VecIter<T>	Vec::iter	&T
KeysIter<K>	HashMap::keys	&K
ValuesIter<V>	HashMap::values	&V
Iter<K, V>	HashMap::iter	(&K, &V)
Iter<T>	HashSet::iter	&T
CharsIter	String::chars, str::chars	Char
SplitIter	String::split	&str
MapIter<I, U>	Iterator::map	U
FilterIter<I>	Iterator::filter	I::Item

Iterators whose Item is a reference (and CharsIter/SplitIter, which borrow their source) are **borrow-carrying types** in the sense of 03-Type-System.md: they hold a live borrow of the collection they iterate and obey all reference rules (generic wrapper storage propagates the borrow, regions are lexical, and no wrapper may outlive its source).

Core Module Structure

```
std/
├── core/           // Core language items
├── collections/    // Data structures
├── io/             // Input/output operations
├── string/         // String manipulation
├── math/           // Mathematical operations
├── mem/            // Memory management utilities
├── error/          // Error handling types
└── prelude/        // Automatically imported items
```

Prelude Module

Automatically imported into every STARK program:

```
// Basic types (no import needed)
Int8, Int16, Int32, Int64, UInt8, UInt16, UInt32, UInt64
Float32, Float64, Bool, Char, String, str, Unit

// Essential traits
trait Copy
trait Clone
```

```
trait Drop
trait Eq
trait Ord
trait Hash
trait Default
trait Display

// Essential enums
enum Ordering {
    Less,
    Equal,
    Greater
}

enum Option<T> {
    Some(T),
    None
}

enum Result<T, E> {
    Ok(T),
    Err(E)
}

// Essential functions
fn print<T: Display>(value: T)
fn println<T: Display>(value: T)
fn panic(message: &str) -> !    // Never returns; see 03-Type-
System.md (Never Type)
```

Core Trait Profile

STD-TRAIT-001. The required Core trait identities are `Copy`, `Clone`, `Drop`, `Eq`, `Ord`, `Hash`, `Default`, `Display`, `Iterator`, `Index`, `IndexMut`, and compiler-owned `Num`, with the required items shown in this document. An implementation claiming the Core profile must provide these canonical items and the primitive/standard-type implementations explicitly required here. Additional traits or implementations are extensions and may not change Core resolution or coherence. The semantic laws of `Eq`, `Ord`, and `Hash` are defined by `TRAIT-LAW-001`; float participation remains owned by C2.9.

Canonical Language Hooks

STD-HOOK-001. A language hook is recognized by canonical standard-item identity, never by an unqualified spelling or source declaration order. The complete Core v1 hook set is:

Hook	Language use
`Copy`	implicit place reads
`Drop` and `drop`	deterministic destruction and explicit consumption

Hook	Language use
`panic`	language trap
`Option` and `Result`	`?` propagation
`Iterator`	`for` protocol
`Index` and `IndexMut`	indexing places/values
`Eq` and `Ord`	generic equality and ordering operators
`Num`	generic primitive numeric operators
`size_of` and `align_of`	target-layout queries

All other APIs—including `Clone`, `Default`, `Display`, `Hash`, collection methods, string methods, math, and I/O—use ordinary name resolution, method selection, trait dispatch, or implementation-provided bodies. `HashMap` may require `Hash`, but that does not make `Hash` a syntax hook. User declarations with a hook's spelling never acquire hook behavior. C2.9 completes the target results of `size_of`/`align_of` and canonical package identity.

Core Module (std::core)

Essential Trait Definitions

```
trait Clone {
    fn clone(&self) -> Self;
}

trait Eq {
    fn eq(&self, other: &Self) -> Bool;
}

trait Ord {
    fn cmp(&self, other: &Self) -> Ordering;
}

trait Hash {
    fn hash(&self) -> UInt64;
}

trait Default {
    fn default() -> Self;
}

trait Display {
    fn fmt(&self) -> String;
}

trait Drop {
    fn drop(&mut self);
}

// Copy is a marker trait (no methods); Copy: Clone.
```

```
// Soundness rules (all-fields-Copy, Copy/Drop exclusivity) are in
// 03-Type-System.md, "Copy and Drop".
trait Copy

// Num is a compiler-known marker trait implemented by exactly the
built-in
// numeric types; it enables arithmetic operators on generic
parameters
// (see 03-Type-System.md, "Operators and Traits"). Not user-
implementable.
trait Num
```

Indexing Traits

The `[]` operator desugars to these traits:

```
trait Index<Idx> {
    type Output;
    fn index(&self, index: Idx) -> &Self::Output;
}

trait IndexMut<Idx> {
    fn index_mut(&mut self, index: Idx) -> &mut Self::Output;
}
```

Range Types

The range operators `..` and `..=` produce these types:

```
struct Range<T> {
    start: T,
    end: T
}

struct RangeInclusive<T> {
    start: T,
    end: T
}

// Ranges over integer types are iterable
impl Iterator for Range<Int32> {
    type Item = Int32;
    fn next(&mut self) -> Option<Int32>;
}
// ... similarly for the other integer types, and for
RangeInclusive
```

Memory Management

```
// Box - heap allocation (implementation-provided; internals
opaque)
struct Box<T> { /* implementation-defined */ }
```

```
impl<T> Box<T> {
    fn new(value: T) -> Box<T>;
    fn into_inner(self) -> T;
}

// Manual memory management
fn drop<T>(value: T)
fn size_of<T>() -> UInt64      // T not inferable: call as
size_of::<Int32>()
fn align_of<T>() -> UInt64     // T not inferable: call as
align_of::<Int32>()
```

Option Type

```
enum Option<T> {
    Some(T),
    None
}

impl<T> Option<T> {
    fn is_some(&self) -> Bool;
    fn is_none(&self) -> Bool;
    fn unwrap(self) -> T;
    fn unwrap_or(self, default: T) -> T;
    fn map<U>(self, f: fn(T) -> U) -> Option<U>;
    fn and_then<U>(self, f: fn(T) -> Option<U>) -> Option<U>;
}
```

Result Type

```
enum Result<T, E> {
    Ok(T),
    Err(E)
}

impl<T, E> Result<T, E> {
    fn is_ok(&self) -> Bool;
    fn is_err(&self) -> Bool;
    fn unwrap(self) -> T;
    fn unwrap_or(self, default: T) -> T;
    fn map<U>(self, f: fn(T) -> U) -> Result<U, E>;
    fn map_err<F>(self, f: fn(E) -> F) -> Result<T, F>;
    fn and_then<U>(self, f: fn(T) -> Result<U, E>) -> Result<U,
E>;
}
```

Collections Module (std::collections)

Vec - Dynamic Array

```
// Implementation-provided; internals opaque
struct Vec<T> { /* implementation-defined */ }

impl<T> Vec<T> {
    fn new() -> Vec<T>;
    fn with_capacity(capacity: UInt64) -> Vec<T>;
    fn push(&mut self, item: T);
    fn pop(&mut self) -> Option<T>;
    fn len(&self) -> UInt64;
    fn capacity(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn get(&self, index: UInt64) -> Option<&T>;
    fn get_mut(&mut self, index: UInt64) -> Option<&mut T>;
    fn insert(&mut self, index: UInt64, item: T);
    fn remove(&mut self, index: UInt64) -> T;
    fn clear(&mut self);
    fn append(&mut self, other: &mut Vec<T>);
    fn extend<I: Iterator<Item = T>>(&mut self, iter: I);
    fn iter(&self) -> VecIter<T>;
    fn as_slice(&self) -> &[T];
}

// Index access
impl<T> Index<UInt64> for Vec<T> {
    type Output = T;
    fn index(&self, index: UInt64) -> &T;
}

impl<T> IndexMut<UInt64> for Vec<T> {
    fn index_mut(&mut self, index: UInt64) -> &mut T;
}
```

`Vec::insert` accepts `0..=len`, shifting later elements right, and traps above `len`; `remove` accepts `0..len`, shifts later elements left, and traps otherwise. `append` moves all elements in order and leaves `other` empty; the borrow rules reject passing the same vector as both receiver and argument. `with_capacity(n)` creates an empty collection with capacity at least `n`; capacity may grow but never affects equality or iteration. Because this constructor has no `Result`, inability to reserve is a classified host/resource failure, not a language trap.

HashMap<K, V> - Hash Table

```
// Implementation-provided; internals opaque
struct HashMap<K, V> { /* implementation-defined */ }

impl<K: Hash + Eq, V> HashMap<K, V> {
    fn new() -> HashMap<K, V>;
    fn with_capacity(capacity: UInt64) -> HashMap<K, V>;
```

```
    fn insert(&mut self, key: K, value: V) -> Option<V>;
    fn get(&self, key: &K) -> Option<&V>;
    fn get_mut(&mut self, key: &K) -> Option<&mut V>;
    fn remove(&mut self, key: &K) -> Option<V>;
    fn contains_key(&self, key: &K) -> Bool;
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn clear(&mut self);
    fn keys(&self) -> KeysIter<K>;
    fn values(&self) -> ValuesIter<V>;
    fn iter(&self) -> Iter<K, V>;
}
```

HashSet - Hash Set

```
// Implementation-provided; internals opaque
struct HashSet<T> { /* implementation-defined */ }

impl<T: Hash + Eq> HashSet<T> {
    fn new() -> HashSet<T>;
    fn insert(&mut self, value: T) -> Bool;
    fn remove(&mut self, value: &T) -> Bool;
    fn contains(&self, value: &T) -> Bool;
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn clear(&mut self);
    fn iter(&self) -> Iter<T>;
}
```

Collection `with_capacity(n)` requests room for at least `n` entries but exposes no exact bucket count, load factor, seed, or growth schedule. Failure to reserve is a classified host/resource failure. Capacity and collision strategy never affect equality, insertion order, returned values, or Drop.

Iteration Order (Core v1)

`HashMap::keys/values/iter` and `HashSet::iter` (and any `for` loop over a `HashMap/HashSet`) MUST visit entries in **first-insertion order**: `insert`ing a key for the first time appends it to the iteration order; `insert`ing an already-present key updates its value without changing its position; `remove`ing a key and later re-`insert`ing it places it at the end (as a new insertion). This requires no bound beyond `Hash + Eq` on the key type (unlike a key-sorted-order rule, which would additionally require `K: Ord` — not part of `HashMap`'s/`HashSet`'s bounds). This is normative and deterministic: two conforming implementations must produce identical iteration order for the same sequence of insertions/removals, regardless of internal storage strategy (see "Performance Notes" below, which describes storage, not iteration order).

STD-HASH-001. `HashMap` and `HashSet` determine key identity exclusively with lawful `Eq` and use `Hash::hash` only to select candidate buckets. Collisions must be resolved by `Eq`; unequal keys with equal hashes remain distinct. Replacing an equal key retains the first stored key and its insertion position. Their observable iteration order is the first-insertion order above and is independent of hash values, collision strategy, capacity, target, and process. Primitive/standard-type hash implementations must be the 64-bit FNV-1a result with offset basis `14695981039346656037` and prime `1099511628211` over this canonical byte encoding:

- integers use their fixed-width little-endian two's-complement/unsigned bits;
- `Bool` is byte 0 or 1; `Char` is its `UInt32` scalar encoding; `Unit` is empty; `String`/`str` are their UTF-8 bytes;
- arrays, tuples, `Vec`, `Option`, and `Result` use domain tags `0x01`, `0x02`, `0x03`, `0x04`, and `0x05` respectively; sequences then encode element count as `UInt64` little-endian; `Option` encodes `None` as variant byte 0 and `Some` as 1, while `Result` encodes `Ok` as variant byte 0 and `Err` as 1; every present component is framed by its encoded byte length as `UInt64` little-endian followed by those bytes.

Floats do not implement `Hash`; maps, sets, resources, references, and borrowed views have no standard `Hash` implementation. These rules make a direct standard `Hash::hash` result backend- and target-independent. User implementations return their body result normally; law violations have the behavior in `TRAIT-LAW-001`.

String Module (std::string)

String Type

```
// Implementation-provided; internals opaque (UTF-8 byte buffer)
struct String { /* implementation-defined */ }

impl String {
    fn new() -> String;               // Empty string
    fn with_capacity(capacity: UInt64) -> String;
    fn from(s: &str) -> String;       // Construct from a string
slice/literal
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn push(&mut self, ch: Char);
    fn push_str(&mut self, s: &str);
    fn pop(&mut self) -> Option<Char>;
    fn clear(&mut self);
    fn chars(&self) -> CharsIter;
    fn bytes(&self) -> &[UInt8];
    fn as_str(&self) -> &str;
    fn into_bytes(self) -> Vec<UInt8>;
    fn substring(&self, start: UInt64, end: UInt64) -> &str;
    fn contains(&self, pattern: &str) -> Bool;
    fn starts_with(&self, pattern: &str) -> Bool;
    fn ends_with(&self, pattern: &str) -> Bool;
    fn find(&self, pattern: &str) -> Option<UInt64>;
    fn replace(&self, from: &str, to: &str) -> String;
    fn split(&self, delimiter: &str) -> SplitIter;
    fn trim(&self) -> &str;
    fn to_lowercase(&self) -> String;
    fn to_uppercase(&self) -> String;
}
```

```
// String literals (&str)
impl str {
    fn len(&self) -> UInt64;
    fn is_empty(&self) -> Bool;
    fn chars(&self) -> CharsIter;
    fn bytes(&self) -> &[UInt8];
    fn to_string(&self) -> String;
    // ... similar methods to String
}
```

TEXT-UTF8-001. Every `String` and `str` value contains valid UTF-8. Literals, file reads, conversions, mutation, concatenation, and native providers must validate before a value becomes observable. Invalid external bytes produce the API's error result; no operation creates a partially valid string or silently substitutes U+FFFD.

TEXT-INDEX-001. String offsets and lengths are unsigned UTF-8 byte offsets. `len`, `find`, `substring`, and split/view boundaries use bytes. Core provides no `string[index]` element operator. `bytes` yields encoded bytes in order; `chars` is the scalar-value API.

TEXT-BOUNDARY-001. A byte offset used as a string boundary must be at zero, at the byte length, or at the first byte of a UTF-8 scalar. `substring` traps when either offset is not a scalar boundary, either is out of range, or start exceeds end. Search/split operations return only valid boundaries.

TEXT-ITER-001. `chars` yields Unicode scalar values in source order; `bytes` yields each UTF-8 byte in source order. Neither normalizes text or combines grapheme clusters. `pop` removes and returns the last scalar, and `push` appends the scalar's UTF-8 encoding.

TEXT-CASE-001. Character classification and `to_lowercase`/ `to_uppercase` use Unicode 15.1 Default Case Conversion, independent of locale. A mapping may expand one scalar to several and preserves their specified order. Core performs no normalization before or after mapping.

`replace(from,to)` scans left-to-right and replaces non-overlapping matches; an empty `from` inserts `to` at every scalar boundary including both ends. `split(delimiter)` preserves trailing empty components for a nonempty delimiter. An empty delimiter yields one component per Unicode scalar and no leading or trailing empty component; splitting an empty string yields no components. `trim` removes the Unicode 15.1 `White_Space` property from both ends only. These operations never normalize or use locale-sensitive matching.

Math Module (std::math)

Basic Operations

```
// Constants
const PI: Float64 = 3.141592653589793;
const E: Float64 = 2.718281828459045;

// Basic functions
```

```
fn abs<T: Num>(x: T) -> T
fn min<T: Ord>(a: T, b: T) -> T
fn max<T: Ord>(a: T, b: T) -> T
fn clamp<T: Ord>(value: T, min: T, max: T) -> T

// Floating point functions
fn sqrt(x: Float64) -> Float64
fn pow(base: Float64, exp: Float64) -> Float64
fn log(x: Float64) -> Float64
fn log10(x: Float64) -> Float64
fn exp(x: Float64) -> Float64

// Trigonometric functions
fn sin(x: Float64) -> Float64
fn cos(x: Float64) -> Float64
fn tan(x: Float64) -> Float64
fn asin(x: Float64) -> Float64
fn acos(x: Float64) -> Float64
fn atan(x: Float64) -> Float64
fn atan2(y: Float64, x: Float64) -> Float64

// Rounding functions
fn floor(x: Float64) -> Float64
fn ceil(x: Float64) -> Float64
fn round(x: Float64) -> Float64
fn trunc(x: Float64) -> Float64

// Random numbers (simple linear congruential generator)
struct Random {
    seed: UInt64
}

impl Random {
    fn new(seed: UInt64) -> Random;
    fn next_int(&mut self) -> UInt64;
    fn next_float(&mut self) -> Float64;
    fn range(&mut self, min: Int32, max: Int32) -> Int32;
}
```

STD-MATH-001. Integer `abs` follows checked integer arithmetic; floating `abs` clears the sign bit. `min`, `max`, and `clamp` dispatch through lawful `Ord` and evaluate arguments once. `floor`, `ceil`, `round` (nearest integer, ties away from zero), and `trunc` produce the exact IEEE result and preserve an already-integral signed zero. Transcendental functions return the IEEE special-case result and a finite result within one ulp of the exact real result (pow within two ulp); their last bit may be target-defined. Domain errors produce NaN rather than a language trap.

STD-RANDOM-001. `Random` is the reproducible 64-bit LCG `state = state * 6364136223846793005 + 1442695040888963407 (mod 2^64)`. `new(seed)` installs `seed`; `next_int` advances once and returns the state. `next_float` advances once and returns the correctly rounded `Float64` value `state / 2^64` in `[0,1)`.

`range(min,max)` requires `min < max`, advances once, and returns `min + state % (max-min)`, hence `min` is inclusive and `max` exclusive; an invalid range traps without advancing.

IO Module (std::io)

Basic IO Operations

```
// Standard streams (implementation-provided generic functions,
not syntax hooks)
fn print<T: Display>(value: T)
fn println<T: Display>(value: T)
fn eprint<T: Display>(value: T)     // stderr
fn eprintln<T: Display>(value: T)   // stderr

// Simple file operations
struct File { /* implementation-defined */ }

impl File {
    fn open(path: &str) -> Result<File, IOError>;
    fn create(path: &str) -> Result<File, IOError>;
    fn read_to_string(&mut self) -> Result<String, IOError>;
    fn write(&mut self, data: &[UInt8]) -> Result<UInt64,
IOError>;
    fn write_str(&mut self, text: &str) -> Result<UInt64,
IOError>;
    fn close(self) -> Result<Unit, IOError>;
}

// Error types
enum IOError {
    NotFound,
    PermissionDenied,
    AlreadyExists,
    InvalidInput,
    Other(String)
}

// Utility functions
fn read_file(path: &str) -> Result<String, IOError>
fn write_file(path: &str, content: &str) -> Result<Unit, IOError>
```

STD-IO-001. The `std-full` profile provides the APIs above and a first-class, non-`Copy` `File` resource whose ownership may be moved but not cloned. Ordinary open/create/read/write/close failures return `IOError`; `NotFound`, `PermissionDenied`, `AlreadyExists`, and `InvalidInput` are used when the host exposes that distinction, otherwise `Other` carries stable human-readable context. Reads validate UTF-8. A successful write reports the number of bytes accepted; callers must handle a short write. Dropping an open file attempts close but cannot surface a new language trap.

STD-FORMAT-001. `Display::fmt` returns valid UTF-8 and is ordinary trait dispatch. `print`/`eprint` append exactly the bytes produced by the argument's `Display` (see PRINT-DISPLAY-001); `println`/`eprintln` append those bytes followed by byte `0x0A`, independent of host newline convention. Successful calls preserve program order. The process contract flushes submitted stdout/stderr before reporting normal return or a language trap; a stream write/flush failure is a host/process failure.

PRINT-DISPLAY-001. `print`, `println`, `eprint`, and `eprintln` are implementation-provided generic functions with the signatures `fn print<T: Display>(value: T)` (and the `println`/`eprint`/`eprintln` analogues). They are **not** syntax hooks: printing dispatches through the argument's `Display` implementation by ordinary trait resolution. For a call `print(value)` (and analogues):

1. Evaluate the argument exactly once.
2. Select the unique coherent `Display` implementation for the argument's type using ordinary trait resolution.
3. Invoke `Display::fmt` exactly once.
4. Print exactly the UTF-8 bytes of the returned `String`.
5. `println`/`eprintln` then append exactly one byte `0x0A`.
6. Destroy the formatting `String` at the call, after its bytes have been submitted.
7. The source argument follows ordinary by-value call ownership semantics (it is consumed by the call; its destructor, if any, runs after the formatted bytes are submitted).
8. If `fmt` traps, the trap propagates normally; no newline and no partial formatting result is printed beyond output already produced by user code before the trap.
9. There is no fallback debug or structural rendering for a type lacking `Display`; such a program is rejected by the checker (E0500).

An implementation MAY keep built-in fast paths for the primitive and standard `Display` types (integers, floats, `Bool`, `Char`, `String`, `str`, `Unit`, `Ordering`, and the containers of them enumerated below), provided their output is observationally identical to the canonical `Display` implementation. A type's internal debug/structural rendering, if any, is a diagnostic facility and is not the language-level `Display`.

Canonical standard `Display` implementations are byte-exact:

- signed/unsigned integers are base-10 ASCII with no leading zeroes and a minus sign only for negative values;
- `Bool` is `true` or `false`; `Char`, `String`, and `str` emit their UTF-8 content; `Unit` is `()`; `Ordering` is `Less`, `Equal`, or `Greater`;
- finite floats use the fewest significant decimal digits that parse back to the same declared IEEE value; among equal-length candidates choose the numerically closest, then the one with an even final digit. Fixed form is used for decimal exponents in `[-4,15]`, otherwise scientific form with one leading digit, lowercase `e`, no exponent `+` or leading zeroes, and at least `.0` or an exponent. Negative zero is `-0.0`; infinities are `inf`/`-inf`; every NaN is `NaN`;
- `IOError` is `NotFound`, `PermissionDenied`, `AlreadyExists`, `InvalidInput`, or `Other(<message>)`; `GenericError` emits its message.

No standard formatting is locale-sensitive or contains padding, grouping, color, debug syntax, or a trailing newline.

Error Module (std::error)

Error Trait

```
trait Error {
    fn message(&self) -> String;
}

// Standard error types
struct GenericError {
    message: String
}

impl Error for GenericError {
    fn message(&self) -> String {
        self.message.clone()
    }
}
```

Note: error *chaining* (a `source()` method returning a trait object) requires `dyn Trait` support, which is a future extension. Core v1 errors carry a message only; richer error types can embed context in their own fields.

Memory Module (std::mem)

Memory Utilities

```
// Memory operations
fn size_of<T>() -> UInt64      // Call with turbofish:
size_of::<Int32>()
fn align_of<T>() -> UInt64     // Call with turbofish:
align_of::<Int32>()
fn swap<T>(a: &mut T, b: &mut T)
fn replace<T>(dest: &mut T, src: T) -> T
fn take<T: Default>(dest: &mut T) -> T
```

Raw-pointer copy operations (`copy`, `copy_nonoverlapping`) require raw pointers and `unsafe`, which are not part of Core v1; they are deferred to a future extension.

Iterator Trait (std::iter)

Basic Iterator Interface

```
trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;

    // Default implementations
```

```
    fn count(self) -> UInt64;
    fn collect<C: FromIterator<Self::Item>>(self) -> C;
    fn map<U>(self, f: fn(Self::Item) -> U) -> MapIter<Self, U>;
    fn filter(self, predicate: fn(&Self::Item) -> Bool) ->
FilterIter<Self>;
    fn fold<B>(self, init: B, f: fn(B, Self::Item) -> B) -> B;
    fn reduce(self, f: fn(Self::Item, Self::Item) -> Self::Item)
-> Option<Self::Item>;
    fn any(self, predicate: fn(Self::Item) -> Bool) -> Bool;
    fn all(self, predicate: fn(Self::Item) -> Bool) -> Bool;
    fn find(self, predicate: fn(&Self::Item) -> Bool) ->
Option<Self::Item>;
}

trait FromIterator<T> {
    fn from_iter<I: Iterator<Item = T>>(iter: I) -> Self;
}
```

Limitation (Core v1): the combinator parameters above are *non-capturing* function types (`fn(...)`). They accept named functions and function references but cannot capture local variables. Capturing closures are a future extension; until then, prefer explicit loops when state must be carried into the operation.

Conversion Traits

Basic Conversion

```
trait From<T> {
    fn from(value: T) -> Self;
}

trait Into<T> {
    fn into(self) -> T;
}

trait TryFrom<T> {
    type Error;
    fn try_from(value: T) -> Result<Self, Self::Error>;
}

trait TryInto<T> {
    type Error;
    fn try_into(self) -> Result<T, Self::Error>;
}

// String conversion
trait ToString {
    fn to_string(&self) -> String;
}

trait FromStr {
    type Error;
```

```
    fn from_str(s: &str) -> Result<Self, Self::Error>;
}
```

STD-CONVERT-001. Numeric `From` exists only for conversions that preserve every source value. Potentially failing numeric conversions use `TryFrom` and the exact `NUM-CAST-001` value/range rules. `FromStr` accepts the corresponding Core literal body without a type suffix or surrounding whitespace and returns `Err` for malformed, non-finite textual forms, overflow, or underflow; it never silently clamps or wraps. Float underflow means a nonzero mathematical input whose correctly rounded result would be zero; subnormal nonzero results are accepted. `as` remains the only trapping numeric conversion syntax.

Essential Trait Implementations

Default Implementations

```
// Copy trait for basic types
impl Copy for Int8 { }
impl Copy for Int16 { }
impl Copy for Int32 { }
impl Copy for Int64 { }
impl Copy for UInt8 { }
impl Copy for UInt16 { }
impl Copy for UInt32 { }
impl Copy for UInt64 { }
impl Copy for Float32 { }
impl Copy for Float64 { }
impl Copy for Bool { }
impl Copy for Char { }
impl Copy for Unit { }

// Clone for all Copy types (blanket implementation)
impl<T: Copy> Clone for T {
    fn clone(&self) -> T { *self }
}

// Eq trait for basic types
impl Eq for Int32 {
    fn eq(&self, other: &Int32) -> Bool { *self == *other }
}

// Ord trait for basic types
impl Ord for Int32 {
    fn cmp(&self, other: &Int32) -> Ordering {
        if *self < *other { Ordering::Less }
        else if *self > *other { Ordering::Greater }
        else { Ordering::Equal }
    }
}
```

Primitive Trait and Operator Matrix

PRIM-TRAIT-001. The impls shown above are illustrative of *form*. The table below is normative for *which* primitive types implement `Eq`, `Ord` and `Hash`, and which operators they admit. It replaces the earlier "similar for other types" wording, which was not a specification.

Primitive	`Eq`	`Ord`	`Hash`	`== !=`	`< <= > >=`
`Int8`…`Int64`, `UInt8`…`UInt64`	yes	yes	yes	yes	yes
`Char`	yes	yes	yes	yes	yes
`String`, `str`	yes	yes	yes	yes	yes
`Bool`	yes	**no**	yes	yes	**no**
`Float32`, `Float64`	**no**	**no**	**no**	yes	yes
`Unit`	no	no	no	no	no

Three rows carry deliberate asymmetries:

- **`Char` is ordered by Unicode scalar value.** `'a' < 'b'` compares scalar values; `Char::cmp` returns the corresponding `Ordering`. This is scalar order, **not** locale-sensitive or linguistic collation, and Core v1 offers no collation facility.
- **`Bool` is `Eq` and `Hash` but not `Ord`.** `true < false`, the other ordered operators, and `Bool::cmp` are compile-time errors. An ordering could be defined, but Core v1 has no use for ordering truth values, and rejecting the operator is clearer than fixing an arbitrary order.
- **Floats admit the comparison operators but implement no trait.** < and == on `Float64` are built-in IEEE 754 operations (`NUM-FLOAT-*`), and remain available. The traits are withheld because IEEE comparison is not an equivalence relation or a total order — NaN is unordered and unequal to itself — so `Float64` cannot satisfy a `T: Eq` or `T: Ord` bound, and cannot be a `HashMap` key.

Operators on primitives have built-in meaning and do not dispatch through these traits (see 03-Type-System, "Operators and Traits"). The trait columns therefore govern *generic bounds* — what `T: Ord` accepts — while the operator columns govern direct use. The two differ only for the float row.

Conformance Profiles

Two standard-library conformance profiles are defined. A conforming implementation MUST state which profile it implements.

STD-PROFILE-001. `core-min` is required for every Core v1 implementation. `std-full` is an optional, indivisible advertised capability: claiming it requires every listed API plus every behavior explicitly stated in this chapter, including file I/O and `Random`. C2.9 freezes language-relevant contracts and the API-availability profile; it does not imply semantics for an edge case this chapter does not state. Such an unstated result is implementation-defined and cannot be used as cross-backend conformance evidence until a later standard-library specification assigns it. A missing host facility prevents the `std-full` claim rather than changing an API's meaning. Extensions may add profiles but may not call them Core v1 or weaken these profiles.

Profile: `core-min` (MVP)

The minimum standard library for Core v1 conformance: - Prelude: primitive types, `Option`, `Result`, `Ordering`, essential traits (`Copy`, `Clone`, `Drop`, `Eq`, `Ord`, `Hash`, `Default`, `Display`, `Iterator`, `Index`, `IndexMut`, `Num`), `print`, `println`, `panic` - `String` and `str` (construction, `len`, `push`/`push_str`, `as_str`, `chars`) - `Vec<T>` (construction, `push`, `pop`, `len`, `get`/`get_mut`, indexing) - `Range`/`RangeInclusive` with integer `Iterator` impls (for `for` loops) - `Box<T>` - `drop`, `size_of`, `align_of`

Profile: `std-full`

Everything in this document: `core-min` plus `HashMap`, `HashSet`, the `Iterator` trait with combinators and all iterator/view types, the full `String` API, the math module, file IO, `std::mem` utilities, the conversion traits, and the full required-trait implementations for standard types.

Items in *neither* profile (informative future work, not required for any Core v1 conformance claim): buffered IO, regular expressions, time/date, threads and concurrency primitives, `Rc`/`Arc`/`RefCell`.

Informative Platform Considerations (Non-Normative)

Cross-platform Abstractions

- File path handling
- Directory operations
- Environment variables
- Process spawning (future)

Behavioral Requirements (Core v1)

- Indexing `Vec<T>` with `[]` MUST perform bounds checking and MUST trap on out-of-bounds access.
- `get`/`get_mut` MUST return `None` for out-of-bounds indices and MUST NOT trap.
- Slicing an array, slice, or `Vec<T>` with a range MUST trap if the range is out of bounds or inverted.
- `String::substring(start, end)` MUST validate UTF-8 boundaries and MUST trap on invalid boundaries or ranges.
- IO functions MUST return `Result` with a non-`Ok` variant on failure and MUST NOT silently ignore errors. ## Conformance A conforming Core v1 implementation MUST implement at least the `core-min` profile, MUST state which profile it implements, and MUST follow the requirements in this

document for every item it provides. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Modules and Packages Specification

Overview

This document defines the module system, visibility rules, and package resolution for the STARK core language. It is normative for Core v1.

Package Layout

A package is a directory containing a `starkpkg.json` manifest at its root.

- The manifest's `entry` field defines the root source file.
- If `entry` is omitted, the default is `src/main.stark`.

All module paths are resolved relative to the package root unless otherwise noted.

Module Declarations

Modules are declared with the `mod` keyword.

```
mod math;

mod inline {
    pub fn add(a: Int32, b: Int32) -> Int32 { a + b }
}
```

File-Based Modules

A declaration `mod name;` has the following candidate files: 1. `name.stark` 2. `name/mod.stark`

The loaded file defines the contents of the module `name`.

MOD-FILE-001. Resolution is relative to the declaring source file's module directory and must remain within the canonical package root after resolving `./..` and symbolic links. Exactly one candidate above must exist; if both exist, the declaration is ambiguous and rejected. One canonical file may define at most one module in a package. Missing files are always errors outside an explicitly identified conformance-harness input mode.

Module Paths

Paths use `::` separators.

- `crate` refers to the package root module.
- `self` refers to the current module.
- `super` refers to the parent module.

Examples:

```
use crate::utils::math::add;
use super::config;
```

MOD-PATH-001. `crate` starts at the current package root, `self` at the current module, and each `super` moves exactly one parent and is rejected at the root. An unqualified first segment follows `NAME-RESOLVE-001`; a dependency alias starts at that dependency's public root. Later segments are resolved only within the preceding module/type namespace. Filesystem paths, checkout locations, and import aliases never participate in public-item identity.

Imports (`use`)

The `use` statement brings names into scope.

```
use crate::utils::math::add;
use crate::utils::{math, io};
use crate::utils::math as m;
use crate::utils::*;
```

Rules: - `use` affects name resolution in the current module only. - `pub use` re-exports the imported name from the current module. - Aliases with `as` are local to the current module.

MOD-USE-001. Nested imports expand as if each leaf were a separate import. An explicit alias introduces only the alias. A glob imports all public names in the selected namespaces but never recursively imports another module's imports. Two imported leaves that introduce the same namespace/name are ambiguous unless they resolve to the same canonical item; an explicit local item also conflicts rather than silently winning. Import processing is independent of declaration and filesystem order.

Visibility

- Items are private to their defining module by default.
- `pub` makes an item visible to parent modules and external modules.
- `priv` explicitly marks an item as private (same as default).

Visibility applies to: - Functions, structs, enums, traits, impl blocks, consts, type aliases, and modules.

MOD-VIS-001. A private item is usable in its defining module and descendant modules only. A public item is externally reachable only through a path whose every module and re-export edge is public. Fields and enum variants follow their declarations' explicit visibility rules; an `impl` cannot make its self type or trait more visible.

MOD-REEXPORT-001. Every type and trait appearing transitively in a public function, constant, field, variant, alias, trait item, or implementation signature must be nameable by consumers through a public canonical path. Private items and dependency items lacking a public re-export are rejected in public API. `pub use` cannot re-export an item the re-exporting module is not permitted to access, and does not create new nominal identity.

Name Resolution Order

Within a module, names are resolved in the following order: 1. Local (lexical) scope — block scopes and function parameters, as defined in 04-Semantic-Analysis.md 2. Items declared in the current module 3. Items brought into scope by `use` 4. Built-in names (prelude)

Unqualified names do not implicitly search parent or crate scopes. Items in parent modules or the crate root require explicit `super::` or `crate::` paths. (Note: *lexical* scopes inside a function body do nest — step 1 — but *module* scopes never nest implicitly.)

Manifest Schema (`starkpkg.json`)

The manifest is a JSON object with the following fields:

Field	Type	Required	Rules
`name`	string	yes	Canonical package identity: `[a-z][a-z0-9_-]*`, 1–64 chars. It is not automatically a source identifier.
`version`	string	yes	Semantic version `MAJOR.MINOR.PATCH` (numeric components; optional `-prerelease` tag).
`entry`	string	no	Package-root-relative path to the root source file. Default `src/main.stark`. MUST exist and MUST be inside the package directory.
`dependencies`	object	no	Map from local import alias to a

Field	Type	Required	Rules
			version constraint string or a *dependency object*.

A dependency object may set `"package"` when the local alias differs from the manifest package name, and has exactly one source form: - `{ "version": "<constraint>", "registry": "<identity>"? }`; - `{ "path": "<relative-path>" }`; or - `{ "git": "<origin>", "rev": "<immutable-revision>", "subdir": "<path>"? }`.

Version constraint syntax: - `"1.2.3"` — exactly that version - `"^1.2.3"` — `>=1.2.3` and `<2.0.0` (compatible-with) - `">=1.2, <2.0"` — comma-separated comparator list (>=, >, <=, <, =), all of which must hold; omitted minor/patch components are zero

Validation: unknown top-level fields are ignored (forward compatibility); a missing/invalid required field, a malformed constraint, a `path` escaping the workspace, an invalid canonical package name, or a dependency alias that is not a non-keyword STARK `IDENTIFIER` is a manifest error and compilation MUST fail. No case or punctuation conversion exists between package names and aliases.

PKG-MANIFEST-001. Manifest input is UTF-8 JSON with one object root. Duplicate keys are errors. `name`, `version`, `entry`, and `dependencies` are validated exactly by this section before source loading. Relative paths are resolved against the manifest directory and confined to the declared workspace after canonicalization. Unknown fields are retained for tooling but have no Core semantics. Core v1 has no conditional dependency features.

Example:

```json
{
  "name": "my-app",
  "version": "0.1.0",
  "entry": "src/main.stark",
  "dependencies": {
    "TensorLib": { "package": "tensor-lib", "version": "^2.1.0" },
    "Utils": { "package": "utils", "path": "../utils" }
  }
}
```

Packages and Dependencies

Dependencies are declared in `starkpkg.json` under `dependencies`.

Resolution order for external packages: 1. Local package source (the current package) 2. Direct dependencies from `starkpkg.json` 3. Standard library package `std`

Dependency modules are accessed via their manifest alias:

```
use TensorLib::tensor::Tensor;
```

Dependency Version Resolution (Core v1)

- Version strings follow semantic versioning.
- If multiple versions satisfy a constraint, the highest version MUST be selected.
- If no version satisfies a constraint, compilation MUST fail with an error.
- The selected source, exact version, and package token MUST be reported in build output.

PKG-RESOLVE-001. A dependency map key is its local import alias. A string value is permitted only when that alias is also a valid canonical package name and names that same package with a registry version constraint. Otherwise a dependency object sets `"package"` to the canonical package name and selects exactly one source:

- `"version"` with optional canonical `"registry"` identity;
- `"path"` to a workspace package; or
- `"git"` with an immutable `"rev"` and optional package-relative `"subdir"`.

Resolution collects the complete graph before selecting versions. All constraints for one logical source, canonical package name, and major-version line are intersected; an empty intersection, source disagreement, alias collision, manifest-name mismatch, or dependency cycle rejects the graph. Aliases affect imports only and never package identity.

PKG-VERSION-001. Semantic versions are ordered by SemVer precedence. Each dependency alias and its constraint must identify exactly one major version line; a constraint spanning two or more majors (for example `>=1.2,<3.0`) is rejected rather than creating multiple instances or choosing one implicitly. Within the identified line, the highest available non-yanked version satisfying every constraint is selected; build metadata does not affect precedence and a tie is broken by exact version string then locked content hash. Prereleases are considered only when a constraint explicitly names a prerelease. With a valid lockfile, its still-compatible exact selection wins without re-resolution.

PKG-IDENTITY-001. A resolved package-instance token is relocation-stable:

- registry: canonical registry identity, package name, exact version, locked content hash;
- git: canonical origin, immutable revision, subdirectory, manifest package name/version, locked content hash;
- workspace path: workspace dependency identity, manifest package name/version, locked content hash—never an absolute checkout path.

A public item identity appends its canonical module path, item name, item kind, and normalized generic arguments. Aliases and re-exports preserve it.

PKG-MULTIVER-001. At most one exact version may be selected for each `(logical source identity, package name, major-version line)`. Different major lines may coexist and have distinct identities, but must be imported through distinct local aliases whose constraints each identify their one major line. Every alias resolves to exactly one package instance. The same exact package instance reached by multiple paths is shared.

PKG-LOCK-001. A reproducible build uses `stark.lock`, generated deterministically from the resolved graph. It records schema version, every package-instance token component, exact dependency edges and aliases, and a cryptographic content hash. Entries are sorted by package token and edges by alias. A missing lockfile may be generated; a stale, incompatible, ambiguous, or hash-mismatching lockfile rejects locked/frozen operation and must never be silently rewritten there.

Standard Library

The standard library is available under the `std` package name:

```
use std::io;
use std::collections::Vec;
```

Cycles

- **Package dependency cycles** — direct (`A → A`) or transitive (`A → B → C → A`) — are a compile-time error.
- **Module declarations** (`mod`) form a tree rooted at the package entry file, so `mod` cycles cannot occur; declaring the same module twice is an error.
- **`use` imports** between modules of the same package MAY be mutually recursive (module A may `use` items from B and vice versa); this is not a cycle error.

MOD-CYCLE-001. Module declarations form a finite tree and duplicate canonical files or ancestor re-entry are rejected. Import cycles are legal because item declarations are collected before import resolution, but a cycle consisting only of unresolved re-export aliases is rejected with a deterministically ordered cycle. Package cycles are always rejected.

Executable and target contract

PROC-MAIN-001. Every package is importable as a library. A build or run request for an executable target uses only the root package and requires exactly one non-generic root item named `main` with no parameters and one of these return types: `Unit`, `Int32`, `Result<Unit, String>`, or `Result<Int32, String>`. Async, overloaded, imported, and dependency `main` items do not qualify. Dependency packages are compiled as libraries even when they declare a function named `main`; a root package containing `main` remains importable as a library. Core v1 has no manifest package-kind or target table.

PROC-EXIT-001. Normal `Unit` and `Ok(Unit)` return status 0. `Int32` and `Ok(Int32)` must be in `0..=255` and return that status; an out-of-range value traps as `invalid-exit-status`. `Err(message)` writes `message` plus LF to stderr and returns status 1. A language trap returns status 101 after its specified diagnostic. Host/process failure has target-defined status and is not a STARK trap. Normal nonzero statuses are normal termination, not traps.

PROC-STREAM-001. At startup stdin is the host-provided byte stream and stdout/stderr are distinct ordered byte streams; Core v1 exposes no stdin read API. Standard output operations obey `STD-FORMAT-001`. All successfully submitted bytes are flushed before normal or language-trap termination. Startup or flush failure is host/process failure. Core adds no terminal encoding, color, carriage return, or platform newline conversion.

LAYOUT-QUERY-001. `size_of<T>` and `align_of<T>` are the only Core layout observations. For every `Sized T` they return positive target-contract values (except a target may report size zero for a zero-sized type), are compile-time/runtime consistent, and satisfy array/field placement needed by safe execution. Discriminant representation, field offsets, niches, pointer values, stack/heap choice, and physical addresses are unobservable.

LAYOUT-ABI-001. Core v1 promises no stable data layout, calling convention, symbol mangling, object format, or cross-package native ABI. Layout-query values may differ between named targets and compiler versions. Interoperation requires a future explicitly versioned native-provider ABI and cannot infer compatibility from equal `size_of`/`align_of` results.

NUM-FLOAT-REPRO-001. Primitive numeric behavior is target-independent as defined by the abstract machine. The named target contract may affect only standard-math last-bit latitude, layout queries, host/process failures, and external I/O—not primitive arithmetic, casts, NaN comparisons, or signed zero.

LIMIT-RESOURCE-001. Allocation, address-space, stack, call-depth, file-descriptor, stream, and other host-resource exhaustion are host/process failures unless an API returns a specified `Result`. Implementations must prevent host undefined behavior and report the classified failure when the host permits; exact capacities are implementation/target-defined.

LIMIT-COMPILER-001. Beyond semantic limits explicitly stated by Core (including 255-byte identifiers, tuple arity 16, and `UInt64` array lengths), an implementation may impose documented finite limits on source size, nesting, items, constant-evaluation work, object size, modules, packages, and dependency solving. Exceeding one rejects compilation with a deterministic limit category and must not masquerade as a syntax/type error, crash, hang, or change a semantic result. A conformance run declares these limits as part of its implementation/target contract.

Errors

- Importing an unknown module or item is a compile-time error.
- Accessing a private item outside its module is a compile-time error.
- Ambiguous imports are a compile-time error unless aliased. ## Conformance A conforming Core v1 implementation MUST follow the requirements in this document. Any deviations or extensions MUST be explicitly documented by the implementation.

STARK Core v1 Future-Extension Boundaries

Status and authority

This chapter is normative for Core v1. It defines compatibility constraints and exclusions, not features that a Core implementation must accept. Syntax or behavior described as future is rejected in Core v1 unless an explicitly enabled, versioned extension owns it.

Reserved syntax and evolution

FUTURE-SYNTAX-001. The reserved words in `01-Lexical-Grammar.md`, lifetime-token space, capturing-lambda delimiters, `dyn`, `unsafe`, `extern`, `async`/`await`, macro invocation and definition space, and attributes are unavailable to Core programs. A reserved word is rejected where an identifier is required even though Core assigns it no grammar production.

A future Core edition or extension may assign these forms only under an explicit language edition or extension version. It may not reinterpret a token sequence that is valid Core v1 with different Core behavior. Unknown edition, extension, attribute, macro, or reserved syntax is rejected rather than ignored. Macros and compile-time code generation are not Core v1 and must not be introduced as an implicit substitute for package, dependency, or build tooling.

Capturing closures and explicit lifetimes

FUTURE-CLOSURE-001. Core v1 function values and `fn(...)` types are non-capturing. A future capturing callable model must distinguish at least:

- consumption on one call;
- mutation through exclusive callable access; and
- repeated calls through shared callable access.

Capture analysis must classify each captured place as moved, shared-borrowed, or exclusive-borrowed; preserve the referent/provenance rules; and make the closure value's own move, Copy eligibility, borrow region, and deterministic Drop behavior derive from its captures. Generic callable bounds may generalize today's non-capturing function parameters, but every valid Core v1 `fn(...)` argument must retain its meaning and dispatch.

Future explicit lifetime parameters may relax Core v1's conservative lexical regions but may not accept a dangling reference or invalidate a currently valid reference. The reserved design space includes lifetime arguments, reference fields, variance, lifetime-bearing associated types, higher-ranked bounds, and explicit erasure at a native ABI boundary. Core v1 accepts none of that syntax and continues to prohibit declared reference fields and lifetime annotations.

Trait objects and runtime vtables are also outside Core v1. A future object-safety and `dyn` design must coexist with static generic dispatch and cannot change an existing impl's coherence, method priority, ownership, or destruction.

Concurrency boundary

FUTURE-THREAD-001. Core v1 program execution is single-threaded. Its ownership and borrowing rules prove aliasing and memory safety only for that execution model; Core makes no claim that all current types are safe to send or share between future threads.

A concurrency extension must separately specify thread-safety marker obligations, atomics and memory ordering, synchronization, shared ownership, thread-local storage, scheduler/process observations, cross-thread destruction, failure/panic propagation, and concurrent native providers. It must preserve Core's exactly-once evaluation and destruction for every execution it accepts and may not introduce a data race into a program accepted as safe.

Native providers, unsafe code, and FFI

FUTURE-FFI-001. Core v1 exposes no public `unsafe`, raw pointer, general FFI, external calling-convention, or arbitrary dynamic-library interface. Host access occurs only through an approved native-provider boundary whose package/artifact metadata identifies:

- provider and artifact identity, origin, integrity hash, and version;
- the versioned ABI and supported targets;
- imported type/ownership/error contracts and provenance;
- required capabilities and host resources; and
- verification status and the safe Core wrapper surface.

Provider internals may use host-unsafe mechanisms, but those mechanisms are not Core operations and cannot weaken Core memory safety, ownership, trap, or capability rules. Pure Core package capabilities must remain statically derivable from the resolved package/provider graph; a provider cannot acquire an undeclared capability through a spelling convention or runtime fallback.

Extension isolation

EXT-ISOLATION-001. Every post-v1 extension has a stable identifier and semantic version and is enabled explicitly in the package/build contract. Without that enablement, its syntax, types, builtins, prelude names, providers, and diagnostics are absent and extension constructs are rejected.

An extension may add explicitly scoped syntax and semantics, but cannot silently change Core tokenization, name resolution, prelude contents, type identity, coherence, ownership, numeric results, standard hooks, package identity, process observations, or diagnostics for a source that uses only Core v1. Packages must declare required extensions and version constraints; unsupported or conflicting requirements reject resolution. There is no silent fallback from an extension operation to a Core approximation or host-specific behavior.

Conformance

A conforming Core v1 implementation must enforce these exclusions and isolation rules. It need not implement any future feature described here.